The Bevi Ranking and Recommendation System

Mathematical specification of `@bevi/ranking` v1.8.0
and the extensions proposed on 2026-10-14

Bevi (Matt & Arjun)*

14 October 2026, revised 24 and 31 October and 10 November 2026

Abstract

Bevi asks a user to compare a beverage ("Bev") against beverages they have already ranked and converts the answers into a personal 0–10 score, a global leaderboard and a prediction of how much a user would like a Bev they have not tried. This document states every formula the system uses, proves the properties the product relies on (termination and correctness of placement, exactness of stored scores, boundedness of predictions), explains the numbers users see (why two ratings of 7.6 and 2.0 display as 5.9), and specifies the extensions agreed in the 14 October 2026 session: category-aware comparisons over a weighted Bev graph (weights from taxonomy, content, text embeddings and flavour profiles, computed into Postgres and mirrored to Neo4j), the Taste Profile, Bev of the day and of the week, likes and an Explore ranking, a Smart Recommend filter for the map, nutrition normalisation and duplicate detection. It closes with the places where the SQL and TypeScript implementations currently differ.

Contents

1 Preliminaries and notation **2**

2 Pairwise placement **3**

3 Score from position **4**

3.1 Exact storage in hundredths 4

4 Statistics and normalisation **5**

5 The Bayesian leaderboard score **6**

*Drafted from `packages/ranking/src`, `DEVELOPMENT-MATT.md` §8 and the second planning session (`CONVERSATION-2.md`). Sections 2–9 describe what is implemented and tested; Section 10 specifies the extensions, several of which have since shipped (its introduction says which). Revised on 2026-10-24 after the M-22 review of this paper against the package and the SQL; the review's correction lists are under M-22 in `TASKS.md`. Revised again on 2026-10-31 for `RANKING_VERSION` 1.4.0 and the category hierarchy, for 1.5.0, which builds the anchored placement of Section 10.1 in the package, and for 1.6.0, in which Layer 2 can read the stored taste neighbours of Section 6.4. Revised on 2026-11-10 for 1.7.0, which adds the graph signals of Section 10.9. Formula changes must bump `RANKING_VERSION` and regenerate the golden vectors in `shared/fixtures`.

6 Predicted score **7**
6.1 Content similarity between Bevs . . . 7
6.2 Layer 1: affinity, content and global . . . 7
6.3 Layer 2: user–user collaborative filtering . . . 8
6.4 Pearson similarity and taste neighbourhoods (v1.3.0) . . . 9
6.5 Blend, confidence and the displayed number . . . 12

7 The recommendation list **12**

8 Cold start **14**

9 Rank on Bevi and streaks **14**

10 Proposed extensions (session of 2026-10-14) **15**
10.1 Category-aware comparisons over a weighted Bev graph . . . 15
10.2 Taste Profile . . . 20
10.3 Bev of the day and Bev of the week . . . 20
10.4 Likes and the Explore ranking . . . 21
10.5 Smart Recommend on the map . . . 21
10.6 Nutrition normalisation . . . 22
10.7 Duplicate detection and merging . . . 22
10.8 One ranking per Bev per user . . . 22
10.9 Graph signals: centrality, communities and bridges (v1.7.0, taste families v1.8.0) . . . 23

11 Implementation notes, divergences and versioning **27**

A Constants **31**

B Symbols **33**

1 Preliminaries and notation

Let U be the set of users, B the set of Bevs and C the set of categories, with $\mathrm{cat} : B \to C$. A *ranking* is a triple $(u, b, s_{u,b})$ with a stored score $s_{u,b} \in [0, 10]$; each user ranks a Bev at most once (Section 10.8). Write $R(u) \subseteq B$ for the Bevs ranked by u, $R^{-1}(b) \subseteq U$ for the users who ranked b, and

$$n_u = |R(u)|, \qquad n_b = |R^{-1}(b)|, \qquad n_{u,c} = |\{b \in R(u) : \mathrm{cat}(b) = c\}|.$$

Every ranking belongs to one of three *buckets* chosen by the user's gut check, each mapped to a score band with integer edges $[\mathrm{lo}, \mathrm{hi}]$:

Bucket	Copy	Band	Database code
LOVED	"Loved it"	$[7, 10]$	3
FINE	"It was fine"	$[4, 7)$	2
DISLIKED	"Didn't like it"	$[0, 4)$	1

The bands are Beli's colour thresholds, so a score's colour and its bucket always agree. Within a bucket a user's Bevs form an ordered list, best first; position $i = 0$ is the best. All constants named in small capitals below are exported from `packages/ranking/src/constants.ts` and listed in Appendix A.

2 Pairwise placement

After the gut check selects a bucket, the new Bev b is placed into the user's ordered list $L = (\ell_0, \ldots, \ell_{n-1})$ for that bucket by binary search on "Which was better?" questions. The search is a pure state machine (`placement.ts`).

Definition 2.1 (Placement state)**.** A state is $(L, \mathrm{lo}, \mathrm{hi}, H)$ with $0 \le \mathrm{lo} \le \mathrm{hi} \le n$, where $[\mathrm{lo}, \mathrm{hi}]$ is the set of candidate *insertion indices* and H the history of answered questions. The initial state is $\mathrm{lo} = 0$, $\mathrm{hi} = n$. While $\mathrm{lo} < \mathrm{hi}$ the current question compares b with ℓ_m, $m = \lfloor (\mathrm{lo} + \mathrm{hi})/2 \rfloor$. The answers are

$$\begin{aligned} \textsc{new_better}: &\quad \mathrm{hi} \leftarrow m, \\ \textsc{existing_better}: &\quad \mathrm{lo} \leftarrow m + 1, \\ \textsc{tie}\ (\text{"Too close to call"}): &\quad \mathrm{lo} \leftarrow m + 1,\ \mathrm{hi} \leftarrow m + 1. \end{aligned}$$

Placement is *done* when $\mathrm{lo} = \mathrm{hi}$; the result is insertIndex $= \mathrm{lo}$. Undo pops H and restores the previous $(\mathrm{lo}, \mathrm{hi})$. An empty list is done immediately at index 0.

Since `RANKING_VERSION` 1.5.0 the same machine also runs the two-phase *anchored* placement of Section 10.1 when `startPlacement` is given a tier per Bev: the anchors are searched first and $[\mathrm{lo}, \mathrm{hi}]$ stays the candidate range in L throughout, so everything below (the result, undo, the log) is shared. Without tiers the machine is exactly this definition.

The number of candidate indices is $c = \mathrm{hi} - \mathrm{lo} + 1$, initially $n + 1$.

Lemma 2.2 (Halving)**.** *Each non-tie answer maps c candidates to at most $\lceil c/2 \rceil$.*

Proof. Let $d = \mathrm{hi} - \mathrm{lo} = c - 1$ and $m = \mathrm{lo} + \lfloor d/2 \rfloor$. After NEW_BETTER the new count is $m - \mathrm{lo} + 1 = \lfloor d/2 \rfloor + 1 = \lfloor (c+1)/2 \rfloor = \lceil c/2 \rceil$. After EXISTING_BETTER it is $\mathrm{hi} - m = d - \lfloor d/2 \rfloor = \lceil d/2 \rceil = \lceil (c-1)/2 \rceil = \lfloor c/2 \rfloor \le \lceil c/2 \rceil$. □

Theorem 2.3 (Termination)**.** *Placement into a list of n Bevs asks at most $\lceil \log_2(n+1) \rceil$ questions. A tie ends the search at once.*

Proof. By Lemma 2.2, after k answers $c \le \lceil (n+1)/2^k \rceil$, which equals 1 as soon as $2^k \ge n + 1$. The code's `maxComparisons(n)` returns the bit length of n, $32 - \mathrm{clz}_{32}(n) = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$ for $n \ge 1$ (and 0 for $n = 0$). Examples: $n = 100 \Rightarrow 7$, $n = 1000 \Rightarrow 10$. □

Definition 2.4 (Consistent oracle)**.** A user is *consistent* with true slot $t \in \{0, \ldots, n\}$ if they answer NEW_BETTER against ℓ_m exactly when $m \ge t$ and EXISTING_BETTER exactly when $m < t$.

Theorem 2.5 (Correctness)**.** *Against a consistent oracle with true slot t, placement returns* insertIndex $= t$.

Proof. Invariant: $\mathrm{lo} \le t \le \mathrm{hi}$. It holds initially. At a question with midpoint m, if $m \ge t$ the oracle answers NEW_BETTER and $\mathrm{hi} \leftarrow m \ge t$; otherwise $m < t$, the answer is EXISTING_BETTER and $\mathrm{lo} \leftarrow m + 1 \le t$. The invariant survives and at termination $\mathrm{lo} = \mathrm{hi} = t$. □

Proposition 2.6 (Log consistency)**.** *Every logged comparison agrees with the final index i^*: a* NEW_BETTER *answer against ℓ_m has $i^* \le m$ (the new Bev ends above ℓ_m), an* EXISTING_BETTER *answer has $i^* \ge m + 1$, and a tie against ℓ_m has $i^* = m + 1$ (directly below the incumbent, which keeps precedence because it was ranked first).*

Proof. Each answer sets $\text{hi} \le m$, respectively $\text{lo} \ge m+1$, and the interval only shrinks afterwards, so the final $i^* \in [\text{lo}, \text{hi}]$ satisfies the inequality. A tie sets both ends to $m+1$. □

`submit_ranking` receives the index, the length of the list that was searched (`expectedBucketSize`; a mismatch is a `conflict`, and the app restarts on a fresh list) and every comparison as a (winner, loser, tie) triple; for a tie the incumbent is recorded as the winner and the flag is set. The server checks the index range and the list length and, since 20261101000200, re-checks every triple against the index before storing anything (Proposition 2.6; Implementation note 11.5): a compared Bev that is not in the list means a stale list and is a `conflict`, an index that disagrees with an answer is a client bug and a `validation` error. A re-rank removes the Bev from its list first, chooses the bucket again and runs the same search.

3 Score from position

Definition 3.1 (Score)**.** In a bucket with band $[\text{lo}, \text{hi}]$, width $w = \text{hi} - \text{lo}$, holding $n \ge 1$ Bevs after insertion, the Bev at position $i \in \{0, \dots, n-1\}$ has

$$\sigma(i, n) = \text{hi} - w\,\frac{i+1}{n+1}. \tag{1}$$

Proposition 3.2. *For all $0 \le i < n$: (i) $\text{lo} < \sigma(i,n) < \text{hi}$; (ii) σ is strictly decreasing in i; (iii) $\sigma(0,1)$ is the band midpoint (8.5, 5.5, 2.0); (iv) $\sigma(i,n) + \sigma(n-1-i,n) = \text{lo} + \text{hi}$ (the spacing is symmetric); (v) ordering a user's whole list by score equals ordering by (bucket, position).*

Proof. (i) follows from $0 < (i+1)/(n+1) < 1$; (ii) and (iii) are immediate; (iv) from $(i+1)+(n-i) = n+1$; (v) from (i), since the bands are disjoint and ordered. □

Inserting or removing a Bev changes n, so every score in that bucket is recomputed ("re-spacing"). The spacing $(i+1)/(n+1)$ was preferred to the midpoint rule $(i+\frac{1}{2})/n$ because it is symmetric under re-indexing and trivial to write in SQL.

3.1 Exact storage in hundredths

Scores are stored as `numeric(4,2)`. To guarantee that TypeScript and Postgres agree on every half-cent, the stored value is computed in integer arithmetic (`storedScoreCents`).

Proposition 3.3 (Exact cents)**.** *Let $D = n+1$ and $N = 100\,(\text{hi}\,D - w\,(i+1))$, both integers. Then $100\,\sigma(i,n) = N/D$ and the half-up rounding of $100\,\sigma$ to an integer is*

$$\text{cents}(i,n) = \left\lfloor \frac{2N + D}{2D} \right\rfloor,$$

which both engines compute exactly. For FINE *and* DISLIKED *the result is capped at $100\,\text{hi} - 1$ so the open upper edge is respected; the cap is first reached at $n = 599$ (*FINE*) and $n = 799$ (*DISLIKED*).*

Proof. $100\sigma = 100\,\text{hi} - 100\,w(i+1)/D = N/D$. Rounding half up is $\lfloor x + \frac{1}{2} \rfloor$ with $x = N/D$, i.e. $\lfloor (2N+D)/(2D) \rfloor$. All quantities are integers below 2^{53}, so IEEE doubles and Postgres integer division are exact. The top position rounds up to hi exactly when $100w/(n+1) \le \frac{1}{2}$, i.e. $n+1 \ge 200w$: $n \ge 599$ for $w = 3$ and $n \ge 799$ for $w = 4$. □

Example 3.4 (Reading a stored score)**.** A stored 7.60 in LOVED means $\frac{i+1}{n+1} = 0.8$ up to the half-cent rounding: the Bev is 4th of 4, 8th of 9, 12th of 14, 16th of 19, … (position $4k$ in a bucket of $5k-1$; no other (i,n) with $n \le 60$ rounds to 7.60). A stored 2.00 in DISLIKED is the middle of an odd-sized bucket: a bucket of one, 2nd of 3, 3rd of 5, and so on. Both numbers from the session are therefore ordinary positions, not anomalies.

4 Statistics and normalisation

All standard deviations are *population* deviations ($\sqrt{\sum(s-\mu)^2/n}$, SQL `stddev_pop`): one rating gives 0 rather than `NULL`, and no inference is ever done on them.

Definition 4.1 (Per-Bev and per-user statistics)**.** For a Bev b: n_b, $\mu_b = \frac{1}{n_b}\sum_{v \in R^{-1}(b)} s_{v,b}$, σ_b. For a user u overall: n_u, μ_u, σ_u over all their stored scores; per category c: $n_{u,c}, \mu_{u,c}, \sigma_{u,c}$.

Definition 4.2 (Effective statistics)**.** With SIGMA_FLOOR $= 1$, MIN_N_USER $= 3$ and the global statistics (μ_G, σ_G) over all stored scores (falling back to $(5, 2.5)$ only when the system has no rankings),

$$(\mu_u^*, \sigma_u^*) = \begin{cases} (\mu_u, \max(\sigma_u, 1)) & n_u \ge 3, \\ (\mu_G, \max(\sigma_G, 1)) & \text{otherwise.} \end{cases}$$

Definition 4.3 (z-score)**.** $z_{u,b} = (s_{u,b} - \mu_u^*)/\sigma_u^*$, always against the user's *overall* effective statistics.

The floor keeps z finite and stops a user whose first three scores are nearly identical from producing enormous z-scores.

Proposition 4.4 (Why not per-category z-scores)**.** *If z-scores were computed against the per-category mean $\mu_{u,c}$, then $\sum_{b \in R(u),\, \mathrm{cat}(b)=c} z_{u,b} = 0$ for every u and c, and every category affinity would be identically zero.*

Proof. $\sum_b (s_{u,b} - \mu_{u,c}) = n_{u,c}\,\mu_{u,c} - n_{u,c}\,\mu_{u,c} = 0$. □

Definition 4.5 (Category affinity)**.** With K_AFF $= 2$,

$$\mathrm{aff}_{u,c} = \frac{\sum_{b \in R(u),\ \mathrm{cat}(b)=c} z_{u,b}}{n_{u,c} + \text{K_AFF}}, \qquad \mathrm{aff}_{u,c} = 0 \text{ when } n_{u,c} = 0. \tag{2}$$

Definition 4.6 (Normalised global score of a Bev)**.** With K_G $= 3$,

$$g_b = \frac{\sum_{v \in R^{-1}(b)} z_{v,b}}{n_b + \text{K_G}}, \qquad g_b = 0 \text{ for a new Bev.} \tag{3}$$

Proposition 4.7 (Shrinkage as a posterior mean)**.** *If $z_1, \dots, z_n \sim \mathcal{N}(\theta, 1)$ independently and $\theta \sim \mathcal{N}(0, 1/K)$, the posterior mean of θ is $\sum z_i/(n+K)$. Hence (2) and (3) are Bayesian estimates of a user's mean preference for a category, and of a Bev's mean normalised quality, under a neutral prior with the weight of K observations.*

Proof. The posterior precision is $n+K$ and the precision-weighted mean is $(n\bar{z} + K \cdot 0)/(n+K)$. □

5 The Bayesian leaderboard score

The number shown on a Bev everywhere in the app (leaderboard, search results, feed cards, Bev detail) is not its plain mean.

Definition 5.1 (Bayesian average)**.** With prior mean m and prior weight $C = \text{BAYES_C} = 3$,

$$\text{bayes}_b = \frac{C\,m + n_b\,\mu_b}{C + n_b} = \frac{C\,m + \sum_v s_{v,b}}{C + n_b}, \qquad \text{bayes}_b = m \text{ when } n_b = 0. \tag{4}$$

Proposition 5.2. *(i)* bayes_b *lies between* m *and* μ_b*; (ii) it is the convex combination* $\lambda_n\mu_b + (1-\lambda_n)m$ *with* $\lambda_n = n_b/(n_b+C)$*, so the Bev's own ratings carry weight* $25\%, 40\%, 62.5\%, 77\%$ *at* $n_b = 1, 2, 5, 10$*; (iii)* $\text{bayes}_b \to \mu_b$ *as* $n_b \to \infty$*; (iv) it is the posterior mean of* θ_b *when* $s_{v,b} \sim \mathcal{N}(\theta_b, \sigma^2)$ *and* $\theta_b \sim \mathcal{N}(m, \tau^2)$ *with* $C = \sigma^2/\tau^2$*.*

Proof. (i)–(iii) are algebra on (4). (iv): the posterior precision is $1/\tau^2 + n/\sigma^2$ and the posterior mean $(m/\tau^2 + n\mu_b/\sigma^2)/(1/\tau^2 + n/\sigma^2)$; multiply numerator and denominator by σ^2. □

Example 5.3 (The 5.9 from the session)**.** Two users ranked the same Bev at 7.6 and 2.0. The plain mean is 4.8; the app showed 5.9. By (4),

$$5.9 = \frac{3\,m + 7.6 + 2.0}{3 + 2} \iff m = \frac{5 \times 5.9 - 9.6}{3} \approx 6.63,$$

so the global mean of all stored scores was about 6.6 at that moment (any $m \in [6.55, 6.71\overline{6})$ gives $\text{bayes} \in [5.85, 5.95)$, which displays as 5.9). The prior, worth three ratings, outweighs the two real ones, which is exactly the design: a single 10/10 cannot top the chart ($(18 + 10)/4 = 7.0$ with $m = 6$). **Nobody's leaderboard rank enters the formula**; the only inputs are the Bev's own scores, their count, and the prior. The leaderboard, the feed, Explore and the Bev-of-the-period card return the live figure rounded to two decimals at the RPC boundary (since 20261101000300), the same rounding `bev_stats.bayes_score`, which the search results and the Bev page read, gets on storage, so one Bev shows one number everywhere and the display's single decimal is taken from the same two (Implementation note 11.6).

Prior mean. The mean of all stored scores in the Bev's category once that category has at least BAYES_CATEGORY_PRIOR_MIN = 30 rankings, else the global mean; one m per board. The overall board, the feed cards, the Bev of the period and the stored per-Bev figure all use the global m (`bayesian.ts`; SQL `bayes_prior`, since 20261019000200, read by every live surface since 20261101000300). With no rankings in the system $m = 5$. Implementation note 11.2 records how the SQL got there.

Eligibility and order. A Bev joins a leaderboard once $n_b \geq \text{MIN_RATERS} = 2$; below that it belongs in a "New" strip. Order: bayes_b descending, then n_b descending, then name ascending in code-point order (SQL `collate "C"`), then id, so the order is total and identical in both engines (`get_bev_leaderboard` since 20261019000200, `leaderboardOrder` by code point since 1.3.0; see Implementation note 11.3).

Tuning rule. C should be roughly the number of ratings at which a Bev's own mean deserves as much trust as the prior; raise C to 5 when the median n_b over rated Bevs passes 10.

6 Predicted score

The prediction $\hat{s}_{u,b}$ ("You'd rate it ∼8.2") is computed in z-units and mapped back through the user's effective statistics. It blends a content layer and a collaborative layer; every function reads one immutable `PredictionSnapshot` (the caller's rankings, every user's statistics, every visible Bev's features and all rankings), so the app, the tests and any future server job give identical output.

6.1 Content similarity between Bevs

Definition 6.1 (Flavour tokens and brand key)**.** $\mathrm{tok}(b)$ is the set of lowercase words of flavour and name (apostrophes dropped), minus a stop list (*original, classic, drink, can, bottle* and unit words such as *oz, ml, pack, mini, regular, large, small*; `FLAVOR_STOPWORDS`) and size tokens (*12oz*, *500ml*, *2l* and bare numbers; `FLAVOR_SIZE_PATTERN`). $\mathrm{brand}(b)$ is the lowercase, whitespace-collapsed brand, or $\perp$ when empty.

Definition 6.2 (Content similarity)**.** With price tiers $\mathrm{tier}(b) \in \{1,2,3,4\}$ and Jaccard index $\mathrm{J}(X,Y) = |X \cap Y|/|X \cup Y|$ (0 when both are empty),

$$\begin{aligned}\mathrm{sim}(b,b') = 0.40\,\mathbf{1}\big[\mathrm{cat}(b) = \mathrm{cat}(b')\big] + 0.30\,\mathbf{1}\big[\mathrm{brand}(b) = \mathrm{brand}(b') \neq \perp\big] \\ + 0.20\ \mathrm{J}(\mathrm{tok}(b), \mathrm{tok}(b')) + 0.10\Big(1 - \tfrac{|\mathrm{tier}(b) - \mathrm{tier}(b')|}{3}\Big).\end{aligned} \tag{5}$$

Pairs with $\mathrm{sim} < \textsc{sim_min} = 0.1$ are dropped from every sum below.

Proposition 6.3. sim *is symmetric and takes values in* $[0,1]$*, with* $\mathrm{sim}(b,b) = 1$ *whenever* b *has a brand and at least one flavour token (*$\mathrm{J}(\emptyset,\emptyset) = 0$ *by convention, so a branded Bev whose name and flavour are all stop words or sizes has* $\mathrm{sim}(b,b) = 0.8$*).*

Proof. Each of the four terms lies in $[0,1]$ and the weights sum to one. □

6.2 Layer 1: affinity, content and global

Definition 6.4 (Layer 1)**.** For user u and candidate b,

$$\mathrm{content}_{u,b} = \frac{\sum_{b' \in R(u)\setminus\{b\}} \mathrm{sim}(b,b')\, z_{u,b'}}{\sum_{b' \in R(u)\setminus\{b\}} \mathrm{sim}(b,b') + 1}, \tag{6}$$

$$\hat{z}^{(1)}_{u,b} = 0.40\ \mathrm{aff}_{u,\mathrm{cat}(b)} + 0.40\,\mathrm{content}_{u,b} + 0.20\,g_b, \tag{7}$$

$$\mathrm{conf}^{(1)}_{u,b} = \min\Big(1, \frac{n_u}{10}\Big) \cdot \min\Big(1, \frac{n_{u,\mathrm{cat}(b)} + \sum_{b'} \mathrm{sim}(b,b')}{4}\Big). \tag{8}$$

The +1 in (6) shrinks the content term toward 0 when little similarity mass exists.

Proposition 6.5. $\hat{z}^{(1)}$ *is in z-units (its weights sum to one and each term is a shrunk mean of z-scores), and* $\mathrm{conf}^{(1)} \in [0,1]$.

Edges mode (v1.4.0, off by default). Called with the option `bevSimilarity: 'edges'`, Layer 1 replaces $\mathrm{sim}(b,b')$ in (6) and (8) by the stored Bev-graph weight w of (25),

$$\mathrm{sim}_E(b,b') = \begin{cases} w(b,b') & \text{if the pair } \{b,b'\} \text{ has a stored edge in } \texttt{bev_edges}, \\ 0 & \text{otherwise,} \end{cases} \tag{9}$$

read from the snapshot's field `edges` (one row per unordered pair, in either orientation). The SIM_MIN rule still applies; it removes nothing while EDGE_MIN_WEIGHT $= 0.3$ and keeps the floor if the edge job's threshold is ever lowered. A pair without a stored edge contributes 0; it is *not* replaced by sim. The two are on different scales (sim ≥ 0.4 for every same-category pair, while a kept w lies in $[0.3, 1]$ and carries embedding and profile terms that sim lacks), so mixing them would make the weighted mean (6) depend on which pairs happened to be pruned. A missing edge was pruned: either $w < 0.3$, or $w \geq 0.3$ but outside both Bevs' $K_E = 20$ heaviest, so in this mode the content term is a K_E-nearest-neighbour item–item average. (Anchored placement, Section 10.1, falls back to sim_T instead, because it needs only an order of tiers, not a weighted mean.) Affinity and g_b are unchanged and the proposition above still holds, since $\text{sim}_E \in [0, 1]$. The mass in (8) becomes $n_{u,\text{cat}(b)} + \sum_{b'} \text{sim}_E(b, b')$, usually smaller than with sim, so $\text{conf}^{(1)}$ saturates later and the $+1$ pulls the content term harder toward 0. A snapshot without the `edges` field falls back to sim, so the option was safe to switch on before `get_prediction_snapshot` returned edges (S-21, shipped as 20261101000100 on 2026-11-01); an empty field means "no edges" and makes every content term 0. The default `'content'` is byte-identical to 1.3.0, and the reason SIMILAR of Section 7 reads whichever content term was used.

6.3 Layer 2: user–user collaborative filtering

User–user filtering was chosen for launch because with about twenty users every pair shares the onboarding starter Bevs, whereas item–item similarity needs co-raters per item pair, which a 380-Bev catalogue and twenty users cannot supply. Switch to item–item at roughly 200 users or when the median number of co-raters per rated item pair reaches 3.

Definition 6.6 (User similarity)**.** Let $\text{common}(u, v) = R(u) \cap R(v)$ and $z_u[\cdot]$ the vector of u's z-scores on it. With CF_MIN_COMMON $= 2$ and CF_SIGNIFICANCE $= 5$,

$$\text{sim}'(u, v) = \begin{cases} \cos\big(z_u[\text{common}], z_v[\text{common}]\big) \cdot \dfrac{\min(|\text{common}(u, v)|, 5)}{5} & |\text{common}(u, v)| \geq 2,\ u \neq v, \\ 0 & \text{otherwise,} \end{cases} \tag{10}$$

where $\cos(x, y) = x \cdot y / (\|x\| \|y\|)$ and is 0 if either vector is zero.

The factor $\min(|\text{common}|, 5)/5$ is significance weighting: two users who agree on two Bevs are not yet neighbours of full weight.

Definition 6.7 (Layer 2)**.** Let $N_K(u, b)$ be the $K = \text{CF_K} = 10$ users $v \neq u$ who ranked b with the largest $\text{sim}'(u, v) > 0$ (ties by user id). Then

$$\hat{z}^{(2)}_{u,b} = \frac{\sum_{v \in N_K} \text{sim}'(u, v)\, z_{v,b}}{\sum_{v \in N_K} \text{sim}'(u, v)}, \qquad \text{supp}_{u,b} = \sum_{v \in N_K} \text{sim}'(u, v), \tag{11}$$

with $\hat{z}^{(2)} = 0$ and supp $= 0$ when no neighbour ranked b.

Proposition 6.8. *$\hat{z}^{(2)}_{u,b}$ is a convex combination of the neighbours' z-scores, hence* $\min_{v \in N_K} z_{v,b} \leq \hat{z}^{(2)}_{u,b} \leq \max_{v \in N_K} z_{v,b}$*, and* $0 \leq \text{supp}_{u,b} \leq K = 10$.

Proof. The weights $\text{sim}'(u, v)/\text{supp}$ are non-negative and sum to one; each $\text{sim}' \leq 1$ and there are at most K of them. □

In score units this is the classical "user's mean plus the weighted average of similar users' deviations": $\mu^*_u + \sigma^*_u \hat{z}^{(2)}_{u,b}$.

6.4 Pearson similarity and taste neighbourhoods (v1.3.0)

Status. Implemented in `graph.ts` (`RANKING_VERSION` 1.3.0, vectors in `graph.vectors.json`) but *not on the live path*: Layer 2 keeps (10) unless it is called with the option `userSimilarity: 'pearson'` or `'stored'`. The taste graph's nightly job (A-16 (2), #133) calls these functions to fill `user_neighbors`. Stage 4's package half is built (1.6.0): Layer 2 called with `userSimilarity: 'stored'` reads the snapshot's optional `userNeighbours` field ("Stored mode" below). The other half shipped on 2026-11-01 as 20261101000100 (S-21 with Stage 4, #147): `get_prediction_snapshot` returns the viewer's rows as `userNeighbors` and the stored Bev edges as `edges`, and `packages/api` maps both onto the snapshot whenever the keys are present. Both modes stay off by default, so nothing reads the rows on the live path until a real export with enough co-raters justifies switching them on. Every formula below is from `docs/graph_db_implementation.md` (approach B, Section 5 and Stage 3), with the open choices marked.

Pearson over the common subset. The z-scores are centred on each user's *overall* mean, so the cosine in (10) rewards two users whose shared Bevs all sit above their own means even when they order them oppositely: $x = (0.5, 1.5)$ and $y = (1.5, 0.5)$ give $\cos(x, y) = 0.6$. Re-centring on the shared Bevs fixes this.

Definition 6.9 (Pearson user similarity)**.** Let $c = \text{common}(u, v)$, $m = |c|$, $x = z_u[c]$ and $y = z_v[c]$, with means $\bar{x}$ and $\bar{y}$ taken over c. Then

$$\rho(u,v) = \begin{cases} \dfrac{\sum_{b\in c}(x_b - \bar{x})(y_b - \bar{y})}{\sqrt{\sum_{b\in c}(x_b - \bar{x})^2 \, \sum_{b\in c}(y_b - \bar{y})^2}} & m \geq 2 \text{ and both sums of squares} > 0, \\ 0 & \text{otherwise,} \end{cases} \tag{12}$$

and its significance-weighted form is

$$\rho'(u,v) = \rho(u,v)\, \min\Big(1, \frac{m}{5}\Big).$$

The constants are those of (10) (CF_MIN_COMMON $= 2$, CF_SIGNIFICANCE $= 5$, and $\min(1, m/5) = \min(m, 5)/5$). With `userSimilarity: 'pearson'`, Definition 6.7 uses ρ' in place of sim' and nothing else in (11)–(19) changes. On the example above, $\rho = -1$.

Proposition 6.10. *ρ is symmetric with values in $[-1, 1]$. It is unchanged when one user's z-scores are replaced by $az + k$ with $a > 0$ (in particular, by adding a constant) and changes sign when $a < 0$. Also $|\rho'| \leq |\rho|$, the factor $\min(1, m/5)$ is non-decreasing in m, and for $m = 2$ with distinct scores $\rho = \pm 1$.*

Proof. Cauchy–Schwarz bounds the ratio by 1 in absolute value. Centring removes k, and a cancels between the numerator and the denominator except for its sign. Two centred points are $\pm\delta/2$, so their ratio is the product of the two signs. □

Zero variance is exact in the code: if a user's z-scores on c are all equal, then $\rho = 0$. (Each vector is shifted by its first entry before centring, so equal entries give an exactly zero sum of squares instead of rounding noise that would correlate at ± 1.) Equal z-scores need equal stored scores, and a user's list only produces those in buckets of 300 or more Bevs.

Taste vectors.

Definition 6.11 (Taste vector)**.** A Bev b carries the attributes $A(b)$: its category, its brand (normalised as in Definition 6.2; none if empty) and its flavour base (none without a profile). These are the graph's `:Category`, `:Brand` and `:Flavor` nodes. An attribute is *kept* when at least TASTE_MIN_ATTR_BEVS $= 2$ catalogue Bevs carry it, so a one-off brand bridges nobody. For a kept a, with $n_{u,a} = |\{b \in R(u) : a \in A(b)\}|$ and $K_T =$ TASTE_K $= 2$,

$$t_u[a] = \frac{\sum_{b \in R(u),\ a \in A(b)} z_{u,b}}{n_{u,a} + K_T}. \tag{13}$$

Like (2), this is a shrunk mean (Proposition 4.7) rather than a sum, so prolific raters do not dominate. It is signed, so a shared dislike counts as agreement and opposite signs push the cosine below zero. Since $K_T =$ K_AFF, $t_u[c] = \mathrm{aff}_{u,c}$ for every kept category c.

Definition 6.12 (Attribute similarity and shared attributes)**.** $\mathrm{sim}_A(u, v) = \cos(t_u, t_v)$, with taste vectors read as sparse vectors (a missing attribute is 0) and $\mathrm{sim}_A = 0$ when either vector is zero. The *bridging attributes* of a pair are the brands and flavour bases a with $|t_u[a]| \geq 0.2$ and $|t_v[a]| \geq 0.2$, of either sign, that the two users reach through *distinct* Bevs: some $b \in R(u)$ and $b' \in R(v)$ with $a \in A(b) \cap A(b')$ and $b \neq b'$. An attribute both users reach only through one Bev they co-rated is that Bev, not a path through a. Categories never bridge: there are only 15 of them, and the doc's candidate query matches `:Flavor` and `:Brand` only. Write $S(u, v)$ for their number (`user_neighbors.shared_attrs`).

Definition 6.13 (Blended user similarity)**.** With $\alpha =$ TASTE_ALPHA $= 0.5$ when $m \geq 2$ and $\alpha = 0$ otherwise,

$$\mathrm{sim}_B(u, v) = \alpha\, \rho'(u, v) + (1 - \alpha)\ \mathrm{sim}^*_A(u, v), \qquad \mathrm{sim}^*_A = \begin{cases} \mathrm{sim}_A & m \geq 2 \text{ or } S(u, v) \geq 2, \\ 0 & \text{otherwise.} \end{cases} \tag{14}$$

The pair is stored with method `blend` if $\alpha = 0.5$ and `attr` if $\alpha = 0$; the Pearson-only arm, $\alpha = 1$, is available only as an explicit option (`pearson`).

The gate on sim_A is the doc's guard against weak multi-hop paths. A pair with fewer than two co-rated Bevs is reachable only by the two-hop path $u \to b \to a \to b' \to v$ with $b \neq b'$. A cosine ignores magnitude, so two faint taste vectors can still point the same way, and the pair must share at least two bridging attributes to count. A pair with two or more co-rated Bevs is reached directly and takes the formula as written. The doc leaves this point open, and the reading here was chosen so that two users who agree perfectly on five co-rated Bevs keep $\mathrm{sim}_B = 1$ however faint their taste vectors are.

Proposition 6.14 (No double counting)**.** sim_B *is symmetric with values in* $[-1, 1]$*, and it lies between* ρ' *and* sim^*_A*. It equals* sim^*_A *when* $m < 2$*, and setting* α *to* 1 *or* 0 *recovers* ρ' *or* sim^*_A *exactly.*

Proof. It is a convex combination of two symmetric quantities in $[-1, 1]$. □

This is how the doc's warning about double counting is met. Agreement on co-rated Bevs reaches both terms, through ρ' and through the taste vectors, which are built from all of a user's rankings. Because the weights are convex, that agreement can never add up past what either term says on its own. Below two co-rated Bevs, $\alpha = 0$ makes the attribute term a substitute for Pearson rather than an addition to it. Consumers must take sim_B as the whole similarity and must not add sim_A again.

Neighbours and reliability. Each user keeps the TASTE_NEIGHBOUR_K $= 25$ users with the largest $\text{sim}_B \geq$ TASTE_SIM_FLOOR $= 0.1$ (ties by id); Layer 2 still uses at most $K = 10$ of them per Bev. Stage 4 weights each neighbour by its reliability $\min(1, n_v/N_0)$ when it reads the rows. That weight is per neighbour and asymmetric, so it is not part of sim_B. The doc leaves N_0 open; $N_0 =$ NEIGHBOUR_RELIABILITY_N $= 15$ is chosen equal to LAMBDA_USER_N, the history at which Layer 2 is fully trusted for the user themself. Under this choice a user fresh from onboarding (10 Bevs) counts 2/3 and one with 3 rankings counts 1/5.

Stored mode (v1.6.0, off by default). Called with the option `userSimilarity: 'stored'`, Layer 2 reads the user's stored neighbours from the snapshot's optional field `userNeighbours`: per user, the rows of `user_neighbors` as (neighbour, sim_B as stored, |common|). With $N(u)$ the list of u, Definition 6.7 uses

$$\text{sim}_S(u, v) = \begin{cases} \text{sim}_B(u, v)\ \min\big(1, n_v/N_0\big) & v \in N(u), \\ 0 & \text{otherwise}, \end{cases} \tag{15}$$

in place of sim′, where n_v is v's number of rankings (the snapshot's `users[v].stats.n`, from v's rankings when v is missing there, and 0 when the snapshot does not know v at all, so such a row weighs 0). Nothing else in (11) changes: the candidates are the listed neighbours who ranked b, a negative stored sim_B is dropped by the rule sim > 0, at most $K = 10$ are used, ties go by user id, and λ (16) reads the new support. Proposition 6.8 still holds, since $\text{sim}_S \leq 1$. Two consequences are the point of the graph. A listed neighbour with one co-rated Bev, whom (10) gives 0, now counts: the attribute bridge of Example 6.15 reaches a prediction. And a rater whom u's list does not name weighs 0 even when (10) is positive. |common| travels with each row for diagnostics and is read by no formula. sim_S is not symmetric: u's list and v's are separate rows, and the reliability is the neighbour's. Fallback: when the snapshot has no `userNeighbours` field (absent or null), or u has no rows (not yet synced by the nightly job, or nobody above the floor), Layer 2 uses (10), as the graph plan's "new users before the nightly sync" requires; so the option is safe to switch on before `get_prediction_snapshot` returns neighbours. The fallback is the cosine, never Pearson, so a fallback prediction is exactly today's. The default `'cosine'` never reads the field and is byte-identical to 1.5.0.

Example 6.15 (The doc's three users)**.** Take Coca-Cola b_1, Pepsi b_2, Coke Zero b_3 (diet soda), Dr Pepper b_4, LaCroix Lime b_5 and Red Bull b_6, with z: $U_1 = (b_1\colon 1.2,\ b_4\colon 0.6,\ b_5\colon -1.3)$, $U_2 = (b_2\colon 1.1,\ b_3\colon 0.8,\ b_5\colon -0.9)$ and $U_3 = (b_4\colon 1.0,\ b_6\colon 0.9)$. The kept attributes are soda (b_1, b_2, b_4), the brand coke (b_1, b_3) and the flavour cola (b_1–b_4); every other attribute covers one Bev. So $t_{U_1} =$ (soda 0.45, coke 0.40, cola 0.45), $t_{U_2} = (0.367,\ 0.267,\ 0.475)$ and $t_{U_3} =$ (soda 0.333, cola 0.333). U_1 and U_2 co-rate only b_5, so $m = 1$ and today's $\text{sim}'(U_1, U_2) = 0$. They share two bridging attributes, coke (b_1 for U_1, b_3 for U_2) and cola (b_1, b_4 and b_2, b_3), so $\text{sim}_B = \text{sim}_A = 0.983$: this is the attribute bridge the graph exists for. U_1 and U_3 also co-rate one Bev, but they bridge on cola alone, so $\text{sim}_B = 0$ even though $\text{sim}_A = 0.847$.

6.5 Blend, confidence and the displayed number

Definition 6.16 (Blend)**.** With $\Lambda_{\max} = \text{LAMBDA_MAX} = 0.7$ and $\text{LAMBDA_USER_N} = 15$,

$$\lambda_{u,b} = \Lambda_{\max} \frac{\text{supp}_{u,b}}{\text{supp}_{u,b}+1} \min\Big(1, \frac{n_u}{15}\Big), \tag{16}$$

$$\hat{z}_{u,b} = (1-\lambda_{u,b})\,\hat{z}^{(1)}_{u,b} + \lambda_{u,b}\,\hat{z}^{(2)}_{u,b}, \tag{17}$$

$$\hat{s}_{u,b} = \text{clamp}\big(\mu^*_u + \sigma^*_u\,\hat{z}_{u,b},\ 0,\ 10\big), \tag{18}$$

$$\text{conf}_{u,b} = \max\big(\text{conf}^{(1)}_{u,b},\ \min(1, \text{supp}_{u,b})\big). \tag{19}$$

Proposition 6.17. $0 \le \lambda_{u,b} \le \Lambda_{\max} \cdot \frac{10}{11} < 0.64$*; with* $\text{supp} = 0$ *the prediction is pure Layer 1, so the Phase 2 (content only) and Phase 3 (collaborative) products share one code path.*

Proof. $\text{supp} \le 10$ by Proposition 6.8 and $x/(x+1)$ is increasing. □

The app displays $\hat{s}$ to one decimal with a tilde prefix and replaces the number by a lock when $\text{conf} < \text{CONFIDENCE_LOCK} = 0.25$. Predictions are withheld entirely while $n_u < 3$ (the recommendations list is empty and the Bev detail shows "Rank 3 Bevs to see your prediction"). A Bev the user has ranked shows their own score instead.

7 The recommendation list

`recommend(snapshot, u, {n, seed})` returns the top n untried Bevs ($\text{RECOMMEND_N} = 20$ by default, capped at 50 by the API).

Definition 7.1 (Candidates and ordering)**.** Candidates are the Bevs in the snapshot not in $R(u)$, not excluded, and not alcoholic unless the user may see alcohol, sorted by id. Each candidate receives a prediction and an *ordering-only* jitter

$$\tilde{z}_b = \hat{z}_{u,b} + \varepsilon_b, \qquad \varepsilon_b \sim \mathcal{N}(0, \text{JITTER_SD}^2),\ \text{JITTER_SD} = 0.15,$$

drawn in id order from a seeded generator (FNV-1a hash of the seed into `mulberry32`; Gaussians by Box–Muller, $\varepsilon = \text{sd}\sqrt{-2\ln(1-U_1)}\cos(2\pi U_2)$). The seed is `userId:localDay`, so a day's list is stable and the next day's shuffles slightly. The displayed score is the un-jittered $\hat{s}$.

Definition 7.2 (Exploration slots)**.** Let $\epsilon = \text{EPSILON} = 0.15$ and let $L = \min(n, |\text{candidates}|)$ be the list length. The list reserves $s = \min(\text{round}(\epsilon n), L)$ slots at the positions

$$p_k = \left\lfloor \frac{(k+1)(L+1)}{s+1} \right\rfloor - 1, \qquad k = 0, \ldots, s-1. \tag{20}$$

For $n = L = 20$: $s = 3$ and $(p_k) = (4, 9, 14)$.

Proposition 7.3. *The slot positions are distinct and lie in* $\{0, \ldots, L-1\}$ *whenever* $s \le L$.

Proof. $p_{s-1} = \lfloor s(L+1)/(s+1) \rfloor - 1 \le L-1$ since $s(L+1)/(s+1) < L+1$; $p_0 \ge \lfloor (L+1)/(s+1) \rfloor - 1 \ge 0$; consecutive positions differ by at least $\lfloor (L+1)/(s+1) \rfloor \ge 1$. □

Definition 7.4 (Exploration pool and draw)**.** Let med be the median of $\{\text{aff}_{u,c} : n_{u,c} > 0\}$ (0 if the user has no rankings). The pool is every candidate b with $g_b \ge 0$ whose category the user has never ranked or has $\text{aff}_{u,\text{cat}(b)} \le \text{med}$. Slots are filled by weighted sampling without replacement with weights $\exp(g_b)$ (a Boltzmann distribution over global quality at unit temperature), using the same generator after the jitter draws. Picks are tagged `isExploration` ("Something different"). Unfilled slots fall back to normal picks; the remaining positions take the non-exploration candidates in decreasing $\tilde{z}$.

Taste families: two options, off by default (v1.8.0). `recommend` has two options that read the snapshot's optional `bevGraph` field, each Bev's *taste family*, PageRank and bridge score from the nightly graph sync (Section 10.9). Write $\phi(b)$ for b's family; a Bev without an entry (created since the last sync) is a family of its own, which cannot coincide with any stored family, and so is a ranked Bev no longer in the snapshot. Write $P_b \in [0,1]$ for its bridge score (34) (0 without an entry). The live engine passes neither option, and *both are unvalidated*: neither constant below has been fitted or tested on real rankings, and the only measurement so far is the synthetic list report of the evaluation harness (`eval -- --lists`), a check that the code runs. With the options omitted, set to their defaults, or set when the snapshot has no `bevGraph` field (the fallbacks `effectiveDiversity`, `effectiveExploration`), the list is byte-identical to 1.7.0's: 6,000 generated lists compared against the 1.7.0 source, 0 differences, including snapshots whose `bevGraph` is present or malformed, which the default never reads. A malformed entry (an empty family id, a negative or non-finite PageRank, a bridge outside $[0,1]$) throws, in community mode only.

Definition 7.5 (Community diversity)**.** Option `diversity: 'community'` (default `'none'`, the order above). Let N be the candidates that are not exploration picks and $L' = L-$(number of exploration picks) the positions they fill. Instead of taking N in decreasing $\tilde{z}$, build the list greedily: with $k_t(\phi)$ the number of the first t picks in family ϕ,

$$b_{t+1} = \operatorname*{arg\,max}_{b \in N \setminus \{b_1,\dots,b_t\}} \Big(\tilde{z}_b - \delta \, k_t\big(\phi(b)\big) \Big), \qquad \delta = \textsc{diversity_penalty} = 0.2, \tag{21}$$

ties to the smaller id, for $t = 0, \dots, L'-1$; $b_1, b_2, \dots$ fill the non-exploration positions in order. Exploration picks neither count in k_t nor move, the jitter draws and the exploration draws are those of the default, and the displayed score is still the un-jittered $\hat{s}$: only the order of the normal picks changes.

δ is in z-units: a second pick from one family must beat the best alternative by 0.2, a third by 0.4. That is the order of JITTER_SD, enough to interleave families whose predictions are close without burying a much stronger candidate.

Proposition 7.6. *(i) b_1 is the normal candidate of largest $\tilde{z}$, so the top normal pick never changes. (ii) If $\delta = 0$, or every family is a singleton, (b_t) is the default order. (iii) A candidate is placed after a candidate of another family with smaller $\tilde{z}$ only when its own family already has $k \geq 1$ picks and its lead is at most δk; a lead above $\delta(L'-1)$ is never overturned.*

Proof. (i) and (ii): $k_0 \equiv 0$, and with $\delta k_t \equiv 0$ (21) is the argmax of $\tilde{z}$ with the same tie rule. (iii): b precedes a only if at that step $\tilde{z}_b - \delta k_t(\phi(b)) \geq \tilde{z}_a - \delta k_t(\phi(a))$; with $\tilde{z}_a > \tilde{z}_b$ this needs $\delta k_t(\phi(a)) > \tilde{z}_a - \tilde{z}_b > 0$ (or equality with a tie), and $k_t \leq L'-1$. □

Within a family every candidate carries the same penalty, so the best one left in a family is the first in that family's $\tilde{z}$ order. The implementation (`diversifyByCommunity`) therefore sorts once, queues the candidates by family and compares only the queue heads at each step: $O(|N| \log |N| + L'C)$ for C families, with no re-sort per pick.

Example 7.7 (A cola lover)**.** Normal candidates with $\tilde{z}$: four colas $1.00, 0.95, 0.90, 0.85$ (one family, whichever the brand or category), teas $0.80, 0.50$, a berry water 0.70. The default lists the four colas first. Under (21): cola 1.00; then cola $0.95 - 0.2 = 0.75$ loses to tea 0.80; cola 0.75 beats tea $0.50 - 0.2$ and berry 0.70; cola $0.90 - 0.4 = 0.50$ loses to berry 0.70; cola 0.50 beats tea 0.30; cola $0.85 - 0.6 = 0.25$ loses to tea 0.30; last the cola. Order: cola, tea, cola, berry, cola, tea, cola. Had the colas been $3.0, 2.9, 2.8, 2.7$ against 2.0 and 1.9, the lead of 0.7 would exceed the penalty 0.6 of the fourth cola and all four would stay first.

Definition 7.8 (Community exploration)**.** Option `exploration: 'community'` (default `'category'`, the pool above). The user's *charted* families are $\Phi_u = \{\phi(b) : b \in R(u)\}$; a ranked Bev without an entry charts only itself. The pool is every candidate b with $g_b \geq 0$ and $\phi(b) \notin \Phi_u$, the uncharted taste families, drawn without replacement with weights

$$e^{g_b}\big(1 + \beta P_b\big), \qquad \beta = \text{EXPLORE_BRIDGE_WEIGHT} = 1. \tag{22}$$

If this pool is empty (the user has ranked in every family, or every uncharted candidate has $g_b < 0$), that call uses the category pool and weights above. The number and positions of the slots (20), the generator and the draw order are unchanged; a pool smaller than the slots leaves the rest to normal picks, as before.

A bridge is the gentlest way into a family the user has not tried: much of its similarity weight goes to families they know. At $\beta = 1$ a Bev with P_b near 1 is drawn up to twice as often as an interior Bev ($P_b = 0$) of the same g_b; with $P_b = 0.9$ against 0 the odds are $1.9 : 1$ (the package's test draws the bridge 65.5% of the time in expectation). The two options are independent: diversity reorders only the normal picks, so the exploration picks and their positions are the same under either diversity setting.

Definition 7.9 (Reason)**.** Each recommendation names the largest contribution to (17): CATEGORY $= (1 - \lambda)\, 0.4$ aff, SIMILAR $= (1 - \lambda)\, 0.4$ content, NEIGHBOURS $= \lambda\, \hat{z}^{(2)}$, POPULAR $= (1 - \lambda)\, 0.2\, g_b$, ties in that order; all zero gives POPULAR.

8 Cold start

New user. Onboarding asks for $k = 10$ starter Bevs: a *core* of the 6 most-rated starters (rating count descending, then name, then id, skipping a starter whose category already has 2 core picks; the same for everyone, which maximises $|\text{common}(u, v)|$ for Layer 2), then 4 drawn with a generator seeded by the user id (so leaving and returning shows the same set), at most 2 per category over the whole set, aiming for at least 5 distinct categories, alcoholic categories excluded under 21. If the per-category cap makes k unreachable it is relaxed. Until the user has 3 rankings, their effective statistics are the global ones (Definition 4.2).

New Bev. No special code: $g_b = 0$ and supp $= 0$, so the prediction is affinity plus content only, and the Bev stays off the leaderboard until $n_b \geq 2$.

9 Rank on Bevi and streaks

Definition 9.1 (Rank points)**.** $\text{points}_u = 1 \cdot \text{bevsRanked}_u + 3 \cdot \text{currentStreak}_u + 2 \cdot \text{bevsCreated}_u$, where bevsCreated_u counts Bevs u added that at least one *other* user has ranked. Followers are deliberately excluded (gameable, unrelated to ranking). Users with no rankings are "Unranked" and do not count toward anyone's rank. Competition ranking: $\text{rank}_u = 1 + |\{v : \text{points}_v > \text{points}_u\}|$; display order is points, then Bevs ranked, then account age, then user id, so the order is total (SQL: `user_rank_points` and `get_user_leaderboard`, with `rank() over (order by points desc)`).

Definition 9.2 (Streak)**.** The day key of a ranking is the calendar date of its timestamp in the user's profile time zone. Given the set D_u of day keys with at least one ranking and today's key t, the current streak starts at t if $t \in D_u$, else at $t - 1$ if $t - 1 \in D_u$ (alive but not yet extended today), else is 0, and counts consecutive days backwards; the longest streak is the longest consecutive run

in D_u. Several rankings on one day count once; re-ranks count (the re-rank day joins D_u and the original day stays); removals never break a streak. D_u is therefore the set of days with a ranking *event*, not the days of the rows that currently exist; since 20261101000200 SQL reads it from the append-only `ranking_events` log (Implementation note 11.4). A weekly variant (ISO weeks) is a constant change.

10 Proposed extensions (session of 2026-10-14)

Status on 2026-10-24. Shipped: the edge weights and the graph pipeline of Section 10.1 (`edges.ts`, `RANKING_VERSION` 1.2.0; `bev_edges` is live on prod), the Taste Profile data of Section 10.2 (A-11, #101), the Bev of the day and week of Section 10.3 (A-05, #96), likes and the Explore ranking of Section 10.4 (A-02 #70, A-04 #113) and the one-ranking rule of Section 10.8. Not built: anchored placement (M-21; waits on the taxonomy, D-04 and A-13), Smart Recommend (Section 10.5, parked), nutrition (Section 10.6) and the duplicate score and merge of Section 10.7 (the S-13 plan). Since then (2026-10-31): the category hierarchy of D-04 (A-13, #132) and the measurement of the co-preference term (A-16 (2), #133) are built, Layer 1 can read the stored weights (edges mode, Section 6, `RANKING_VERSION` 1.4.0), off until the evaluation harness has real data, anchored placement is built in the package (`placementTiers`, `startPlacement` with `tiers`, `RANKING_VERSION` 1.5.0, M-21), with the app's wiring of it (`ui/anchored-placement`) the remaining step, and Layer 2 can read the stored taste neighbours (stored mode, Section 6.4, `RANKING_VERSION` 1.6.0), off until the harness has real data and `get_prediction_snapshot` returns them. Since 2026-11-10 the package also computes centrality, communities and bridges over the Bev graph (Section 10.9, `RANKING_VERSION` 1.7.0), for the graph sync, and (1.8.0) the flavour projection the stored taste families are clustered on; `recommend()` can read the families and bridges behind two options that ship off (Section 7). Each subsection gives the definition precisely enough to code and test; where a choice was open it now records the decision.

10.1 Category-aware comparisons over a weighted Bev graph

Motivation. External feedback: ranking a Bev against a Bev of a very different kind feels wrong ("I'd always take mid Indian food over top-tier American food"). The agreed answer: *the first comparisons are against the most similar Bevs the user has ranked*, same subcategory first, then adjacent categories; categories become hierarchical and Bevs live in a graph whose edges carry similarity weights.

Taxonomy. Let T be the two-level category tree of D-04, in Postgres since A-13 (#132): the 15 categories are roots (`categories.parent_id` null) and 60 leaves sit under 13 of them (e.g. soda → cola, lemon-lime, …; `water` and `sports` are unsplit). A Bev keeps its root in `bevs.category_id`, so $\mathrm{cat}(b)$, and with it the content similarity (5), affinity and every per-category statistic, still means the root. Its leaf is the nullable `bevs.leaf_category_id`, null when no finer placement is known (the unsplit roots, and a user-created Bev until it is relabelled); write

$$\mathrm{leaf}(b) = \texttt{leaf_category_id} \mathbin{??} \texttt{category_id},$$

the leaf when there is one and the root otherwise. Write $\mathrm{depth}(x)$ for the depth of a node and $\mathrm{lca}(x, y)$ for the lowest common ancestor. Depth is measured from an implicit root above the 15 roots: the implicit root has depth 0, a root category depth 1 and a leaf depth 2 (`treeSimilarity`, and the `treeDepth` line of the vectors' header). Two Bevs under different roots therefore meet only

at the implicit root and have $\mathrm{sim}_T = 0$; sibling leaves under one root have $\mathrm{sim}_T = 2 \cdot 1/(2+2) = 0.5$; a leaf against its own root (one Bev placed on the parent) has $2 \cdot 1/(2+1) = 2/3$; the same leaf, or the same unsplit root, has 1.

Definition 10.1 (Tree similarity, Wu–Palmer)**.**

$$\mathrm{sim}_T(b, b') = \frac{2\ \mathrm{depth}(\mathrm{lca}(\mathrm{leaf}(b), \mathrm{leaf}(b')))}{\mathrm{depth}(\mathrm{leaf}(b)) + \mathrm{depth}(\mathrm{leaf}(b'))} \in [0, 1], \tag{23}$$

equal to 1 for the same leaf and decreasing as the common ancestor moves toward the root. The tree term is measured at the leaf while the content term (5) inside w keeps the root (the edge job since #132): handing the leaf to both would count the taxonomy twice inside w and cost the content term its coarse same-shelf signal (Coca-Cola against Fanta would drop from 0.40 to 0 on its category term).

Definition 10.2 (Embedding and profile similarity)**.** Let $e_b \in \mathbb{R}^{384}$ be the unit-norm text embedding of "brand, name, flavour, category" (the `gte-small` model that ships with Supabase Edge Functions) and $\mathrm{sim}_{\mathrm{emb}}(b, b') = \max(0, e_b \cdot e_{b'})$. Let every Bev carry a *flavour profile* $\pi_b = (\mathrm{base}, \mathrm{carb}, \mathrm{caff}, \mathrm{sweet}, \mathrm{dairy}, \mathrm{alc})$: a categorical base family (cola, citrus, coffee, matcha, ...), carbonation, dairy and alcohol as binary flags, caffeine and sweetness on an ordinal 0..3 scale. Then

$$\begin{aligned} \mathrm{sim}_{\mathrm{prof}}(b, b') = {} & 0.40\, \mathbf{1}[\mathrm{base}_b = \mathrm{base}_{b'}] + 0.15\, \mathbf{1}[\mathrm{carb}_b = \mathrm{carb}_{b'}] \\ & + 0.10\, \mathbf{1}[\mathrm{dairy}_b = \mathrm{dairy}_{b'}] + 0.05\, \mathbf{1}[\mathrm{alc}_b = \mathrm{alc}_{b'}] \\ & + 0.15\Big(1 - \tfrac{|\mathrm{caff}_b - \mathrm{caff}_{b'}|}{3}\Big) + 0.15\Big(1 - \tfrac{|\mathrm{sweet}_b - \mathrm{sweet}_{b'}|}{3}\Big). \end{aligned} \tag{24}$$

Both similarities are symmetric and lie in $[0, 1]$.

Definition 10.3 (Edge weight)**.**

$$w(b, b') = \alpha\ \mathrm{sim}_T + \beta\ \mathrm{sim} + \delta\ \mathrm{sim}_{\mathrm{emb}} + \eta\ \mathrm{sim}_{\mathrm{prof}} + \gamma\ \mathrm{sim}_{\mathrm{cop}}, \qquad \alpha + \beta + \delta + \eta + \gamma = 1, \tag{25}$$

every term evaluated at (b, b'), with the content similarity sim of (5) and the learned *co-preference* term

$$\mathrm{sim}_{\mathrm{cop}}(b, b') = \max\big(0,\ \cos\big((z_{v,b})_{v \in V},\ (z_{v,b'})_{v \in V}\big)\big), \qquad V = R^{-1}(b) \cap R^{-1}(b'),$$

and $\mathrm{sim}_{\mathrm{cop}} = 0$ while $|V| < 3$. As built in A-16 (2) (#133): $z_{v,b}$ is v's z-score from all of v's rankings (`zScoresFromRankings`), only active Bevs contribute, and the value is stored in `bev_edges.components.cop` while $\gamma =$ `EDGE_WEIGHTS.cop` is still 0, so no stored weight depends on it yet. Ruling (2026-10-31): a negative cosine is clamped to 0 rather than mapped through $(1+r)/2$, so that 0 always means "no evidence of similarity", the same convention as $\mathrm{sim}_{\mathrm{emb}}$; under $(1+r)/2$ an uncorrelated pair would score 0.5, and the term would add weight to pairs about which the data says nothing. Decided 2026-10-14: the first version uses $(\alpha, \beta, \delta, \eta, \gamma) = (0.25, 0.15, 0.30, 0.30, 0)$ and switches γ on (renormalising the others) once co-raters exist. *When γ is switched on.* Renormalising the other four weights by $(1 - \gamma)$ shrinks every pair without positive co-preference evidence, which today is almost every pair (a median of one co-rater per pair, #122), by the fraction γ. The switch must therefore re-derive the 0.3 floor and the sanity bounds below ($w \geq 0.8$ for Coca-Cola–Pepsi, $w < 0.4$ for Coca-Cola–LaCroix), or it prunes edges for lack of data rather than lack of similarity. Before switching, consider a significance weight $\min(1, |V|/5)$ on $\mathrm{sim}_{\mathrm{cop}}$, as Layer 2 weights users (10), and a candidate arm for pairs co-rated by at least 3 users, which the candidate set below can miss; and move `copSimilarity` from `scripts/graph/edges-core.ts` into `edges.ts` with golden

vectors, like the other four terms. Each of these is a `RANKING_VERSION` bump. Weights are computed over a candidate set, the union of same category, same brand and the 50 nearest by embedding cosine (`scripts/graph/build-edges.ts`); each term is rounded to six decimals before w is formed, so the stored components reproduce the stored weight exactly. A pair is kept when either endpoint ranks it among its $K_E = \text{EDGE_TOP_K} = 20$ heaviest with $w \geq \text{EDGE_MIN_WEIGHT} = 0.3$ (`topEdges`; ties on w go to the smaller id in code-point order, which for lowercase-hex uuids is also `uuid` byte order and `collate "C"`), so a popular Bev can have more than 20 incident edges. A pair with no stored edge falls back to sim_T alone in anchored placement (M-21); Layer 1's edges mode gives it 0 instead (9).

Example 10.4 (Coca-Cola and Pepsi)**.** Same leaf (cola) gives $\text{sim}_T = 1$; content gives 0.5 (same category and price tier, different brands, no shared flavour tokens); two colas embed close together, sim_{emb} near 0.9; identical profiles give $\text{sim}_{\text{prof}} = 1$. Hence $w \approx 0.25 + 0.075 + 0.27 + 0.30 \approx 0.90$. For Coca-Cola and a plain sparkling water the tree, content and profile terms all fall (different leaf, no shared tokens, different base and sweetness), and w drops below 0.4. The content term alone would have put the two colas at 0.5, which is why the embedding and profile terms were added. These are the golden-vector figures (0.895 and 0.315, `edges.vectors.json`). The first real run against prod (2026-10-24, #121; 383 Bevs embedded with `gte-small`, 4669 edges stored) measured Coca-Cola–Pepsi $w = 0.9001$ and Coca-Cola–LaCroix Lime $w = 0.3809$, both inside the sanity bounds $w \geq 0.8$ and $w < 0.4$ that the job prints. Coca-Cola and LaCroix sit under different roots, so their tree term is 0 with or without the hierarchy, and the second bound holds only while their embedding cosine stays at or below 0.88; the measured weight corresponds to a cosine of about 0.82. Measuring sim_T at the leaf (#132) changes no formula and leaves both figures unchanged (Coca-Cola and Pepsi share the leaf *cola*); the weights that move are those of pairs under one root on different leaves, which lose exactly $\alpha(1 - 0.5) = 0.125$, and of a leaf against its own root, which lose $\alpha/3 \approx 0.083$.

Profiles for user-created Bevs (D-11, decided 2026-10-19). A Bev with no hand-labelled profile copies the profile of the nearest seeded Bev in the same category by embedding cosine (brand match, then id, as tie-breaks), marked *inferred* with its source and the cosine, provided the cosine is at least 0.6; below that no profile is written and the Bev's sim_{prof} term is 0, so it is weighed by the tree, content and embedding terms alone (`infer_bev_flavor_profile`, 20261023000200).

Remark 10.5 (Where the graph lives). Decided 2026-10-14, built in #115, #117, #118 and #121 (A-15, A-16 (1)). Postgres is the system of record for Bevs, users and rankings, and it is also where the weights are computed: the `embed` Edge Function writes e_b to `bev_embeddings` (pgvector, 384-d, on every Bev insert or update, plus a backfill), and the nightly job `scripts/graph/build-edges.ts` (04:50 UTC) reads Bevs, categories, profiles and embeddings, weighs the candidate pairs with `@bevi/ranking` and replaces `bev_edges` in one transaction, stamping `RANKING_VERSION`. No step of that pipeline touches Neo4j, so the ranking flow and the recommender read weights from Postgres and never wait on the graph. Neo4j AuraDB Free (`Bev`, `Category`, `Brand` and `Flavor` nodes; `SIMILAR_TO` edges carrying w and its components; pseudonymous `RANKED` edges; a vector index over e_b) is the mirror of `bev_edges`, where similarity is inspected and where the co-rating neighbourhoods behind sim_{cop} will be computed; the mirror sync (A-16 steps 6, 8 and 9) is a later increment, the instance sits on no request path, and the nightly `(:Meta)` write is what keeps a Free instance from pausing (`docs/graph.md`). The centrality, community and bridge signals that sync computes over these edges are defined in Section 10.9 (`RANKING_VERSION` 1.7.0).

Anchored placement. Let $L = (\ell_0, \ldots, \ell_{n-1})$ be the user's bucket list and b the new Bev. Partition L into *tiers* by similarity to b. The original proposal said "T_1 = same leaf category, T_2 = same parent, ..., or more generally by decreasing $w(b, \ell_i)$ bands"; the rule built in `RANKING_VERSION` 1.5.0 (M-21, `placementTiers`) is, for a bucket Bev ℓ with w the stored `bev_edges` weight of $\{b, \ell\}$ when there is one,

$$\text{tier}(b, \ell) = \begin{cases} 1 \text{ (same leaf)} & \text{cat}(b) = \text{cat}(\ell), \text{ both leaves known and equal,} \\ 2 \text{ (same root)} & \text{cat}(b) = \text{cat}(\ell) \text{ otherwise,} \\ 3 \text{ (stored edge)} & \text{cat}(b) \neq \text{cat}(\ell) \text{ and } w \geq \text{EDGE_MIN_WEIGHT,} \\ \text{none} & \text{otherwise,} \end{cases} \tag{26}$$

where "leaf known" means `leaf_category_id` is not null. Three decisions are folded into (26). *Unknown kind.* A Bev whose leaf is unknown is tier 2 against everything under its root, in either direction, and two Bevs of unknown kind under one root are tier 2 with each other: this is D-04 §3's rule that T_1 is read as empty when the leaf is unknown ("kind unknown" never reaches the same-leaf tier). *Parent-level and siblings together.* Wu–Palmer puts a leaf against its own root at 2/3, above sibling leaves at 1/2; the tiers do not separate them. The 2/3 is the expected value of not knowing the leaf, not evidence that the Bev is closer, so anchoring on it ahead of a known sibling would ask the first question about the Bev whose kind is least certain; a tier of its own would split the anchor set and lengthen the search (Remark 10.7) for no like-for-like gain; and after the D-04 §7 corrections no catalogue Bev sits on a split root, so only user-created Bevs would populate it. *Weights as a third tier, not as bands.* A stored edge reaches across roots where the taxonomy cannot (Coca-Cola for a new Diet Coke when no diet soda is ranked: cola and diet cola are leaves of different roots, $\text{sim}_T = 0$, but w carries brand, embedding and profile). The taxonomy ranks above it because it is exact, while the existence of a stored edge depends on the edge job's pruning (K_E, the 0.3 floor); beyond the floor the value of w is not used, so a re-run of the job with changed weights cannot reorder the anchors within a tier, and the "decreasing w bands" generalisation stays available as a later version with the data to choose the band edges. Without weights the rule is the taxonomy alone (tiers 1 and 2), which is what the app sends when `bevs.similarities` is unavailable. Because the anchors are the most similar *non-empty* tier, what matters is the set each tier defines, not its number; two Bevs under the unsplit root `water` are tier 2, and they are the anchors for a new water all the same.

Let $A = (a_1, \ldots, a_k)$ be the members of the most similar non-empty tier, in list order, at positions $p_1 < \cdots < p_k$ in L.

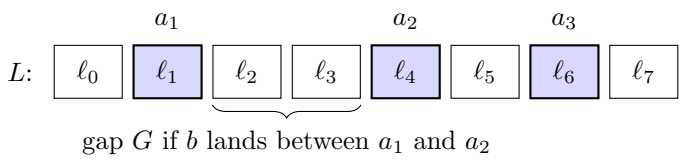


Figure 1: Anchors (shaded) are the user's ranked Bevs most similar to the new Bev. Phase 1 binary-searches the anchors only; Phase 2 searches the gap between the two anchors the new Bev landed between.

As built (`placement.ts`, 1.5.0). Algorithm 1 runs inside the state machine of Section 2 rather than as two separate searches. `startPlacement` takes an optional `tiers` array, one entry per bucket Bev (`null` for none; the Bev being re-ranked is filtered out of both), and the state gains `anchored`

Algorithm 1 Anchored placement of b into L

1: **if** A is empty **then return** ordinary placement (Section 2) on L
2: **end if**
3: **Phase 1.** Run the placement state machine on the anchor list A (length k). ▷ every question is like-for-like
4: **if** a TIE against a_j ended Phase 1 **then return** $p_j + 1$
5: **end if**
6: Let $j^* \in \{0, \dots, k\}$ be the resulting slot among anchors.
7: $\mathrm{lo} \leftarrow (j^* = 0 \ ? \ 0 : p_{j^*} + 1)$; $\mathrm{hi} \leftarrow (j^* = k \ ? \ n : p_{j^*+1})$ ▷ candidate indices in L
8: Let $G = (\ell_{\mathrm{lo}}, \dots, \ell_{\mathrm{hi}-1})$ be the gap, $g = \mathrm{hi} - \mathrm{lo}$ Bevs, all non-anchors.
9: **Phase 2.** Run the placement state machine on G, giving slot $r \in \{0, \dots, g\}$. ▷ cross-category, only inside the gap
10: **return** $\mathrm{lo} + r$

`= {positions, lo, hi}`: the anchor positions $p_1 < \cdots < p_k$ and the candidate *anchor slots* $[\mathrm{lo}_A, \mathrm{hi}_A] \subseteq [0, k]$. The state's own $[\mathrm{lo}, \mathrm{hi}]$ remains the candidate insertion range in L throughout: after every Phase 1 answer it is recomputed as $\mathrm{lo} = (\mathrm{lo}_A = 0 \ ? \ 0 : p_{\mathrm{lo}_A} + 1)$ and $\mathrm{hi} = (\mathrm{hi}_A = k \ ? \ n : p_{\mathrm{hi}_A+1})$, the two lines of line 5 applied to the current slot range, so the invariant $0 \le \mathrm{lo} \le \mathrm{hi} \le n$ of Section 2 holds in both phases, Phase 2 is literally the plain machine on that range once $\mathrm{lo}_A = \mathrm{hi}_A$, and `placementResult` is one function (`insertIndex` = lo). A tie against a_j in Phase 1 sets $\mathrm{lo} = \mathrm{hi} = p_j + 1$ (line 3). Every history step records both ranges, so `undo` restores a Phase 1 state from inside Phase 2, reopening Phase 1 at the question that was answered. `currentPrompt.maxSteps` is $\lceil \log_2(k+1) \rceil + \lceil \log_2(g_{\max} + 1) \rceil$ with $g_{\max}$ the widest gap (before the first anchor, between two, or after the last), constant for the placement so the progress indicator does not jump when Phase 2 starts; `placementPhase` tells the app which phase a question belongs to. With no `tiers`, or with no tiered Bev, the state is 1.4.0's field for field (the fallback of line 1); with every Bev in one tier, $p_j = j - 1$ makes the two ranges the same numbers and the questions and the result coincide with the plain search.

Theorem 10.6 (Properties of anchored placement)**.** *Against a consistent oracle with true slot t in L:*

(i) the returned index is t, and lies in $\{0, \dots, n\}$ for any sequence of answers;
(ii) every logged comparison agrees with the final index in the sense of Proposition 2.6;
(iii) at most $\lceil \log_2(k+1) \rceil + \lceil \log_2(g+1) \rceil$ questions are asked: at most $\lceil \log_2(k+1) \rceil$ in Phase 1, all of them before any Phase 2 question and each against an anchor, and at most $\lceil \log_2(g+1) \rceil$ in Phase 2, each against a non-anchor inside the gap; if the gap is empty no cross-tier question is asked at all;
(iv) every Phase 1 question, the first included, is against a Bev of the most similar non-empty tier. It is the middle *anchor of the current slot range, not a "most similar" anchor: within a tier the members are equally similar by construction, and binary search gives the order among them no meaning. M-21's definition of done is read accordingly ("every Phase 1 question is like-for-like" in place of "the first question is the most similar anchor").*

When $k = 0$ or $k = n$ the questions and the result are those of plain placement on L.

Proof. (i) Consistency on L implies consistency on the subsequence A with true anchor slot j^* such that $p_{j^*} < t \le p_{j^*+1}$ (with the conventions $p_0 = -1$, $p_{k+1} = n$), so Phase 1 returns j^* by Theorem 2.5 and $t \in [\mathrm{lo}, \mathrm{hi}]$. The gap G is exactly $\ell_{\mathrm{lo}}, \dots, \ell_{\mathrm{hi}-1}$ and the oracle is consistent on it with true slot $t - \mathrm{lo}$, so Phase 2 returns $r = t - \mathrm{lo}$. For arbitrary answers, Phase 1 yields $j^* \in [0, k]$

hence $0 \le \mathrm{lo} \le \mathrm{hi} \le n$, and Phase 2 yields $r \in [0, g]$, so $\mathrm{lo} + r \in [0, n]$. (ii) A Phase 1 answer against a_j constrains j^* relative to j, hence the final index relative to p_j in the same direction; a Phase 2 answer constrains r relative to a position inside the gap. (iii) and (iv) follow from Theorem 2.3 applied to each phase and from the construction of A: Phase 1 asks only about members of A and finishes before Phase 2 starts; the gap lies strictly between two consecutive anchors (or before the first, or after the last), so it contains none. For $k = n$ every $p_j = j - 1$ and the anchor slots are the insertion indices; for $k = 0$ the fallback applies. □

Remark 10.7 (Cost). In the worst case ($k \approx n/2$, a wide gap) anchored placement asks about twice as many questions as plain binary search; in the typical case the gap between two adjacent anchors is short and the total is close to $\lceil \log_2(n+1) \rceil$. The bound can exceed the plain one by a question or two even with few anchors: two lattes at the top of a list of 100 give $\lceil \log_2 3 \rceil + \lceil \log_2 99 \rceil = 2 + 7 = 9$ against 7. Decided (1.5.0): `startPlacement` does *not* fall back to the plain search when `maxAnchoredComparisons` exceeds `maxComparisons`(n), because few same-kind Bevs in a long list is exactly the case the feature exists for (a threshold at +1 would switch it off there), the bound is a worst case that a consistent user rarely reaches, and the trade is a product one; the app can read `currentPrompt.maxSteps` against `maxComparisons`(n) and pass no `tiers` if it ever wants a cap. The product requirement, that the like-for-like questions come first, is met exactly. The result keeps the existing contract (`insertIndex`, `expectedBucketSize`, `comparisons`), so `submit_ranking` is unchanged. The app needs each bucket Bev's root and leaf (`rankings.listBucketBevs` and `bevs.leaves`, contract 2.8.0; a Bev missing from `leaves` is treated as leaf unknown) and, for tier 3, the new Bev's stored weights to the bucket (`bevs.similarities`); a failed or slow similarities call degrades to the taxonomy rule, and a failed leaves call to the plain search, never to a blocked flow. That wiring is `ui/anchored-placement`. An alternative, per-category lists with no cross-category questions at all, would give up the single comparative order behind (1); anchored placement is preferred for that reason.

10.2 Taste Profile

The session defined the Taste Profile as "your average score of cafe drinks could be this number out of 10" and "you prefer these kinds of drinks".

Definition 10.8 (Taste Profile)**.** For user u the profile is the table $\{(c, n_{u,c}, \mu_{u,c}, \mathrm{aff}_{u,c}) : n_{u,c} \ge 1\}$ ordered by $\mu_{u,c}$ descending. The display uses the raw means (what the user recognises); recommendations keep using aff (comparable across users). A preference statement "prefers c over c'" is shown when $n_{u,c}, n_{u,c'} \ge 3$ and $\mu_{u,c} - \mu_{u,c'} \ge 1$.

The table `user_category_stats` holds exactly this, written by `refresh_user_category_stats` after every ranking write and nightly for everyone (A-11, 20261019000100; means and aff stored to three decimals), and read through `get_taste_profile`, which returns the rows with $n_{u,c} \ge 1$ ordered by $\mu_{u,c}$ descending, then the category's sort order, then its id. The pgTAP suite checks the stored rows against the "five rankings" case of `user-stats.vectors.json`.

10.3 Bev of the day and Bev of the week

Definition 10.9 (Period winner)**.** Fix the zone America/New_York ("strictly New York City"). For a period P (a local calendar day, or an ISO week) let R_P be the rankings created in P and $B_P = \{b : \exists\, r \in R_P \text{ with Bev } b\}$ the Bevs ranked in P. The winner is

$$b_P^* = \arg\max_{b \in B_P} \big(\mathrm{bayes}_b,\ |R_P(b)|,\ n_b,\ -\mathrm{name}(b)\big) \tag{27}$$

in lexicographic order, then the Bev's id, where $|R_P(b)|$ is the number of rankings of b in P and bayes_b is the Bev's current Bayesian score (4) with the global prior. Candidates are the active Bevs the viewer may see (the age gate applies) and there is no MIN_RATERS gate, so one ranking today can win. Decided 2026-10-18 (D-02): if $B_P = \emptyset$, show the winner of the most recent non-empty period with its date, flagged as not current (`bev_of_period_at`, 20261018000100, which the pgTAP suite pins on a fixed clock).

Remark 10.10. Using the current bayes_b rather than only the period's scores keeps the card consistent with the leaderboard the user can open, and lets a single ranking today surface a Bev while the prior still tempers it. The alternative, a period-local shrunk mean $(Cm + \sum_{r \in R_P(b)} s_r)/(C + |R_P(b)|)$, rewards what was *said* today rather than what is *good*; it is one line to switch.

10.4 Likes and the Explore ranking

Definition 10.11 (Like count and scope)**.** $\ell_p \in \mathbb{N}$ is the number of distinct users who liked post p. For user u with follow set $F(u)$, the Explore scope is the second hop only, $E(u) = \big(\bigcup_{v \in F(u)} F(v)\big) \setminus (F(u) \cup \{u\})$: strangers whose posts the feed does not already show. Decided 2026-10-19 (D-01, D-15): while Bevi has fewer than 20 profiles the scope widens to every user outside $F(u) \cup \{u\}$, and Explore posts are read-only (no likes, counts or comments are shown).

Definition 10.12 (Hotness)**.** For a post p created at t_p, viewed at time t, with age $h = (t - t_p)$ in hours and gravity $\gamma_{\text{hot}} = 1.5$,

$$\text{hot}(p, t) = \frac{\ell_p + 1}{(h + 2)^{\gamma_{\text{hot}}}}. \tag{28}$$

Explore lists the posts of $E(u)$ from the last 14 days by hot descending, then post id descending, with t the server's clock (never a caller parameter, so the like counts cannot be read off by moving it) and h clamped at 0 for a future-dated post (`explore_hotness` and `get_explore`, 20261019000400). "Bevs getting the most likes" over a window W is $\text{pop}_W(b) = \sum_{p \in W,\ \text{bev}(p) = b} \text{hot}(p, t)$; not built, since no screen asks for it yet.

Proposition 10.13. hot *is increasing in likes and decreasing in age; a post with ℓ likes outranks a post with ℓ' likes that is Δ hours younger iff $(\ell + 1)/(\ell' + 1) > \big((h + 2)/(h - \Delta + 2)\big)^{\gamma_{\text{hot}}}$. With $\gamma_{\text{hot}} = 1.5$ a day-old post needs about 2.5 times the likes (plus one) of a twelve-hour-old post to stay level.*

Proof. Direct from (28); the numeric example is $((24 + 2)/(12 + 2))^{1.5} \approx 2.53$. □

10.5 Smart Recommend on the map

Parked by the session's guardrail ("too many features") but specified so it can be built when wanted.

Definition 10.14 (Venue score)**.** A venue v carries a set of Bevs $M(v)$ (for chains, a shared assortment per chain plus venue-specific items). With category filter $F \subseteq C$ and untried set $\bar{R}(u) = B \setminus R(u)$,

$$\text{score}(u, v; F) = \max\big\{\, \hat{s}_{u,b} : b \in M(v) \cap \bar{R}(u),\ \text{cat}(b) \in F,\ \text{conf}_{u,b} \geq 0.25 \,\big\}, \tag{29}$$

or $-\infty$ if the set is empty. Smart Recommend shows exactly the venues with $\text{score}(u, v; F) \geq \theta$, with $\theta = 7$ (the LOVED threshold), and sorts them by score.

The maximum (rather than a mean) answers the question the user asks on a map, "is there something here I would love?", and inherits the lock rule of Section 6 so low-confidence guesses never light up a venue.

10.6 Nutrition normalisation

A vision model reads the label photo into per-serving facts $x = (x_1, \dots, x_d)$ (calories, fat, sodium, carbohydrate, sugars, protein, caffeine, ...) together with the serving volume V in millilitres.

Definition 10.15 (Per 100 ml)**.** $x_i^{(100)} = 100\, x_i / V$. Per-container values multiply x_i by the number of servings per container.

Definition 10.16 (Acceptance checks)**.** An extraction is accepted when all values are non-negative, sugars $\leq$ carbohydrate, added sugars $\leq$ sugars, $V > 0$, and the energy identity holds within tolerance:

$$|\text{calories} - (4\,\text{carbohydrate} + 4\,\text{protein} + 9\,\text{fat})| \leq \max(10,\ 0.15 \cdot \text{calories}).$$

Otherwise the result is stored as *needs review* and nothing is shown.

Nutrition is shown on the Bev's own page to whoever opens it, never in the feed or in rankings (the session: "just for the user themselves to see").

10.7 Duplicate detection and merging

Definition 10.17 (Duplicate score)**.** For a query q (name, brand, category, vessel) and an existing Bev b, with trigram similarity tri (`pg_trgm`) over the normalised "brand name" text and its aliases,

$$d(q,b) = \max\big(\text{tri}(q,b),\ \max_{a \in \text{aliases}(b)} \text{tri}(q,a)\big) + 0.15\,\mathbf{1}[\text{brand}(q) = \text{brand}(b)] + 0.10\,\mathbf{1}[\text{cat}(q) = \text{cat}(b)]\,, \tag{30}$$

and $d(q,b) = 1$ whenever both carry the same UPC. "Did you mean..." shows the top five with $d \geq 0.3$ (the `pg_trgm` default); a user-flagged duplicate with $d \geq 0.6$ is queued for merge review, below that for manual review.

Definition 10.18 (Merge)**.** Merging b_{src} into b_{dst}: for each user who ranked both, keep the ranking with the better (bucket, position) and remove the other; re-point remaining rankings, aliases and Want to Try rows; recompute statistics; set b_{src} to status *merged* with `merged_into` $= b_{\text{dst}}$.

Proposition 10.19. *A merge preserves, for every user, the relative order of all Bevs other than the merged pair, and every stored score remains inside its band.*

Proof. Removal of one ranking followed by re-spacing is a monotone relabelling of positions in that bucket (Proposition 3.2(ii)); other buckets are untouched; band membership is Proposition 3.2(i). □

Status: the "Did you mean..." step is live as `find_similar_bevs`, which uses the trigram part of (30) (the maximum over the Bev's text and its aliases) with the category as a filter rather than a bonus; the brand and category bonuses, the duplicate report and `merge_bev` are the S-13 plan.

Vessels (can, glass bottle, plastic bottle) are *not* distinct Bevs and must never enter the name: the natural key is (category, brand, name, flavour). A genuinely different product (Mexican Coca-Cola, cane sugar, glass) is its own Bev with its own icon.

10.8 One ranking per Bev per user

The ranking relation is a partial function $U \times B \rightharpoonup [0, 10]$: a user has or has not ranked a Bev, and if so holds exactly one score. Re-ranking *updates* this value (and re-spaces the old and new buckets); it never appends. Consequently n_b counts people, not drinks, and "how many times you drank it" is deliberately not a quantity in this system (session decision; the database enforces it with a unique constraint on (user, Bev)).

10.9 Graph signals: centrality, communities and bridges (v1.7.0, taste families v1.8.0)

Status. Implemented in `network.ts` (`RANKING_VERSION` 1.7.0; the flavour projection of Definition 10.30 in 1.8.0; vectors in `network.vectors.json`). This subsection follows the Bev graph of Section 10.1; it is numbered last only so that the subsections after Section 10.1, which code and SQL comments cite by number, keep their numbers. Three signals per Bev, for the nightly job that mirrors `bev_edges` into Neo4j and stores them in `bev_graph_stats`:

- *centrality*, a weighted PageRank: how representative a Bev is of its neighbourhood (many similar Bevs, heavily weighted, lead to it);
- *community*, Louvain modularity clustering: data-driven *taste neighbourhoods* that cut across the hand-built category tree. Since 1.8.0 the community the sync stores is the Bev's *taste family*: Louvain over the flavour projection of Definition 10.30, not over the similarity graph itself, which mostly rediscovers the categories (the measurement after Definition 10.30). A family holds Coke, Diet Coke and Pepsi together whatever their categories;
- *bridge*, the participation coefficient over the similarity graph with the taste families as the partition: how much of a Bev's similarity weight goes to Bevs of *other* families, i.e. a gateway from one family into another.

No prediction consumes them. The prediction snapshot carries them in an optional field `bevGraph` (per Bev: the family's opaque id, the PageRank and the participation coefficient), which nothing read in 1.7.0; since 1.8.0 `recommend()` reads the family and the bridge, never the PageRank, behind two options that ship off (Definitions 7.5 and 7.8). Every formula lives here and nowhere else: there is no SQL twin, the sync job calls these functions, and Neo4j's own graph-algorithm library (a paid product) is not used.

Setting and order. Let $G = (V, E, w)$ be the graph the sync sees: V the Bevs, E the stored pairs, $w_{ij} = w_{ji} > 0$ the weight (25); $n = |V|$, $N(i)$ the neighbours of i,

$$s_i = \sum_{j \in N(i)} w_{ij} \quad \text{(strength, the weighted degree)}, \qquad m = \tfrac{1}{2} \sum_{i \in V} s_i = \sum_{\{i,j\} \in E} w_{ij}.$$

Node ids are compared with $\prec$, Unicode code-point order (`compareCodePoints`, SQL `collate "C"`, Implementation note 11.3). Every sum over nodes runs in $\prec$ order and every sum over $N(i)$ in $\prec$ order, so each function is a function of the graph alone: not of the order of `nodes` or `edges`, nor of an edge's orientation. The summation orders are part of the contract and are spelled out in the vectors' header rules (`networkGraphRule` and the five that follow it). All six functions reject the same malformed inputs: a duplicate or empty id, an endpoint that is not a node, a self-loop, a pair given twice (in either orientation), a non-finite weight, or $w \le 0$. The empty graph is valid and gives empty results.

Definition 10.20 (Weighted PageRank)**.** Let $d \in [0, 1)$ be the damping PAGERANK_DAMPING $= 0.85$ and $D = \{i : s_i = 0\}$ the isolated Bevs. For p in the simplex Δ_n let

$$F(p)_j = \frac{1 - d}{n} + d\Big(\sum_{i \in N(j)} p_i \frac{w_{ij}}{s_i} + \frac{1}{n} \sum_{i \in D} p_i \Big), \tag{31}$$

the random walk $P_{ij} = w_{ij}/s_i$ followed with probability d, a uniform teleport otherwise, and the mass on isolated Bevs spread uniformly (back to themselves included). Start from $p^{(0)} = (1/n, \ldots, 1/n)$ and iterate $p^{(t+1)} = F(p^{(t)})$ until $\|p^{(t+1)} - p^{(t)}\|_1 < \tau =$ PAGERANK_TOLERANCE $= 10^{-10}$ or

$t+1 = T =$ PAGERANK_MAX_ITERATIONS $= 200$, a safety rail that the defaults never reach (Remark 10.22). The result is the last iterate divided by its own sum, $\pi = p/\sum_j p_j$. Each term of the inner sum is evaluated as $(p_i w_{ij})/s_i$.

Proposition 10.21. *F maps Δ_n to itself and is a contraction with constant d in $\|\cdot\|_1$. Hence it has a unique fixed point π^*, $\|p^{(t)} - \pi^*\|_1 \le 2d^t$, and the returned vector is non-negative and sums to 1. If $D = \{z\}$, then $\pi^*_z = (1-d)/(n-d)$. Scaling every weight by $\lambda > 0$ leaves F, hence π^*, unchanged.*

Proof. $\sum_j F(p)_j = (1-d) + d\big(\sum_{i\notin D} p_i \sum_{j\in N(i)} w_{ij}/s_i + \sum_{i\in D} p_i\big) = (1-d) + d\sum_i p_i = 1$, and $F(p) \ge 0$. For $p, q \in \Delta_n$, $F(p) - F(q) = d\,M(p-q)$ with $M_{ji} = w_{ij}/s_i$ for $i \notin D$ and $M_{ji} = 1/n$ for $i \in D$; every column of M sums to 1, so $\|Mx\|_1 \le \|x\|_1$ and Banach's theorem applies, with $\|p^{(0)} - \pi^*\|_1 \le 2$. For a single isolated z, $N(z) = \emptyset$ and $\pi^*_z = (1-d)/n + d\,\pi^*_z/n$. Scaling multiplies w_{ij} and s_i alike. □

Remark 10.22 (The tolerance, not the cap, ends the iteration). Since F is a d-contraction, the L_1 step of iteration $t+1$ is $\|p^{(t+1)} - p^{(t)}\|_1 \le d^t \|p^{(1)} - p^{(0)}\|_1 \le 2d^t$ on every graph, and $\|p^{(t+1)} - \pi^*\|_1 \le \frac{d}{1-d}\|p^{(t+1)} - p^{(t)}\|_1$. At $d = 0.85$, $2d^t < 10^{-10}$ once $t > 145.9$, so the iteration stops at the latest at step 147, below $T = 200$, and the result is within $\frac{d}{1-d}\tau \approx 6\cdot 10^{-10}$ of π^* in L_1. The bound is attained on a bipartite component (a path, a star), where the walk has eigenvalue -1 and the iterates oscillate around π^* with ratio d; even there the tolerance decides: the 3-node path returns 0.4864864865 for its centre, $18/37$ to ten digits. T binds only when a caller passes a smaller `maxIterations` (or a larger d or smaller τ), and then the capped vector is still deterministic. The vectors pin the default runs, which a cap of 1000 reproduces bit for bit.

For an undirected graph and $d \to 1$, $\pi^*_i \to s_i/2m$ on a connected graph, so PageRank is a smoothed strength: it also rewards being similar to Bevs that are themselves central, and the teleport keeps a weakly connected Bev above 0. A Bev with high π is a good *representative* of its neighbourhood.

Definition 10.23 (Modularity)**.** For a partition $c : V \to \mathbb{Z}$ with communities C and resolution $\gamma \ge 0$ (LOUVAIN_RESOLUTION $= 1$),

$$Q_\gamma(c) = \sum_C \Big[\frac{W_C}{m} - \gamma\Big(\frac{S_C}{2m}\Big)^2\Big], \qquad W_C = \sum_{\substack{\{i,j\}\in E\\ i,j\in C}} w_{ij}, \quad S_C = \sum_{i\in C} s_i, \tag{32}$$

and $Q_\gamma = 0$ when $m = 0$. W_C/m is the share of the weight inside C; $(S_C/2m)^2$ is that share for a random graph with the same strengths, so Q measures how much more within-community similarity there is than chance. Above $\gamma = 1$ smaller communities are preferred, below it larger ones.

$Q_\gamma \le 1$, the single community has $Q_\gamma = 1 - \gamma$, all singletons have $-\gamma\sum_i (s_i/2m)^2$, and Q_γ is invariant under scaling w (every term is a ratio of weights).

Lemma 10.24 (Move gain)**.** *Let i be a node of strength k_i (at an aggregated level, its internal weight ℓ_i counted twice in k_i) and $C \not\ni i$ a community; write $k_{i,C}$ for the weight between i and C (not counting ℓ_i). Then*

$$Q_\gamma(i \text{ in } C) - Q_\gamma(i \text{ alone}) = \frac{k_{i,C}}{m} - \gamma\,\frac{S_C\,k_i}{2m^2} =: \Delta Q(i, C). \tag{33}$$

Proof. Joining C adds $k_{i,C}$ to W_C and k_i to S_C, and removes i's own term, whose ℓ_i/m reappears inside W_C. The null-model terms change by $-\gamma\big[(S_C + k_i)^2 - S_C^2 - k_i^2\big]/4m^2 = -\gamma\,S_C k_i/2m^2$. □

Definition 10.25 (Louvain communities, deterministic)**.** The two-phase method of Blondel et al. (2008), with every choice fixed:

(i) *Level* 0*:* every node its own community.

(ii) *Local moves:* visit the nodes in $\prec$ order, in passes, until a whole pass moves nothing. Visiting i: take it out of its community A, then evaluate $\Delta Q(i,\cdot)$ of (33) (with S_C the strength of C without i) for *its own community* $A \setminus \{i\}$ *first*, possibly empty, and then for each community holding a neighbour of i, in ascending order of *representative*, the $\prec$-smallest member id. A candidate replaces the best so far only if its gain is larger by more than $\varepsilon = \text{LOUVAIN_TIE_EPSILON} = 10^{-12}$. So a tie keeps i where it was, otherwise it goes to the earlier candidate, and rounding noise between two equal gains never decides a move.

(iii) *Aggregation:* each community becomes one node whose id for ordering is its representative, the weight between two communities becomes one edge, and the internal weight becomes a self-loop ℓ that counts in k but not in $k_{i,C}$. Return to (ii) on the new graph.

(iv) *Stop* at the first level whose local moves move nothing.

The output lists each community's members in $\prec$ order, and the communities by size descending, then by first member in $\prec$ order; $c(i)$ is the index of i's community in that list. Isolated Bevs are singletons; a graph without edges is all singletons with $Q = 0$.

Theorem 10.26 (Termination and monotonicity)**.** *In exact arithmetic every move raises Q_γ by more than ε. The local moves of a level therefore stop after finitely many passes, at most n levels run, and the result has Q_γ at least that of all singletons. The result depends only on G and γ, and it is unchanged when every weight is scaled by $\lambda > 0$.*

Proof. By Lemma 10.24, moving i from A to B changes Q_γ by $\Delta Q(i,B) - \Delta Q(i, A \setminus \{i\}) > \varepsilon$. Since Q_γ is bounded above by 1, at most $(1 - Q_{\text{start}})/\varepsilon$ moves can happen. A move always goes to a non-empty community, so the number of non-empty communities never increases and the first move of a level lowers it; a level that moves anything leaves fewer nodes for the next. Aggregation preserves Q_γ, so Q never decreases from the all-singletons start. Every step reads its inputs in $\prec$ order. Under scaling, every gain is a ratio of weights, so the comparisons come out the same (up to rounding far below ε). □

In floating point the gains of equal candidates differ by rounding of order 10^{-16}, four orders of magnitude below ε. The implementation also stops with an error after 10,000 passes within one level, a guard against a bug and not a formula input. Louvain is a heuristic. It finds a good partition, not necessarily the one of maximum modularity, and it shares modularity's resolution limit (Fortunato and Barthélemy, 2007): communities whose internal weight is small compared with $\sqrt{m}$ can be merged into larger ones. γ is the lever if the taste neighbourhoods come out too coarse.

Definition 10.27 (Participation coefficient)**.** Given a partition c and $s_{i,C} = \sum_{j \in N(i),\, c(j)=C} w_{ij}$ (the own community included),

$$P_i = 1 - \sum_C \Big(\frac{s_{i,C}}{s_i}\Big)^2 \quad (s_i > 0), \qquad P_i = 0 \quad (s_i = 0), \tag{34}$$

after Guimerà and Amaral (2005). Small negative rounding is clamped to 0.

$P_i = 0$ exactly when all of i's similarity mass goes into one community. The implementation sums $s_{i,C}$ in the same order as s_i, so this holds bit for bit. If i touches κ_i communities then $P_i \le 1 - 1/\kappa_i < 1$, with equality for an even split. P_i is unchanged by scaling w. A Bev with high P_i is a *bridge*, a natural "if you like these, try those" step out of a neighbourhood.

Definition 10.28 (Community hub)**.** The hub of a community C is

$$h(C) = \arg\max_{i \in C} \sum_{j \in N(i) \cap C} w_{ij},$$

its member with the greatest within-community strength (compared as the exact doubles summed in $\prec$ order over $N(i)$), ties going to the $\prec$-smallest id. A singleton is its own hub. The hub identifies the community. The sync derives the community's public id from $h(C)$, hashed so that an age-gated hub's Bev id never reaches a client, and that id is opaque: clients compare it for equality only. Because the public id follows the hub, this rule is part of the contract (`communityHubs`, the header rule `communityHubRule`).

Example 10.29 (Two cliques and a weak bridge)**.** Two 4-cliques $\{a_1, \dots, a_4\}$ and $\{b_1, \dots, b_4\}$ with unit weights, joined by one edge $a_4 b_1$ of weight 0.1: $m = 12.1$, $s = 3$ inside and $s_{a_4} = s_{b_1} = 3.1$. Louvain returns the two cliques, with

$$Q_1 = 2\Big(\frac{6}{12.1} - \Big(\frac{12.1}{24.2}\Big)^2\Big) = \frac{12}{12.1} - \frac{1}{2} \approx 0.49174.$$

Only the bridge endpoints participate, $P_{a_4} = P_{b_1} = 1 - (3/3.1)^2 - (0.1/3.1)^2 = 0.6/9.61 \approx 0.06243$. PageRank puts them slightly above the interior nodes (≈ 0.12773 against ≈ 0.12409; the eight values sum to 1), and the hubs are a_1 and b_1 (all within-strengths are 3, so the smallest id wins). A ring of four triangles joined corner to corner gives four communities with $Q_1 = 4(3/16 - (8/32)^2) = 1/2$ and $P = 4/9$ at every corner that carries a ring edge.

Taste families (v1.8.0). The definitions above apply to any weighted graph. The sync runs PageRank and the participation coefficient on the similarity graph G, and Louvain on a second graph built from the Bevs' flavours.

Definition 10.30 (Attribute projection, Adamic–Adar)**.** Let V be the Bevs and X a set of attributes, with $b \sim x$ when Bev b carries attribute x, and $\deg(x) = |\{b : b \sim x\}|$. The one-mode projection $G_F = (V, E_F, w^F)$ joins two Bevs that share an attribute, with the weight of Adamic and Adar (2003)

$$w^F_{ab} = \sum_{x:\ a \sim x,\ b \sim x} \frac{1}{\ln \deg(x)}, \tag{35}$$

so a common attribute says little and a rare one a lot. An attribute carried by one Bev joins nothing and is skipped ($\ln 1 = 0$); every other term is positive and finite. Attributes are processed in $\prec$ order, each attribute's Bevs in $\prec$ order, and each pair's weight is accumulated from 0 in that attribute order. The output lists V in $\prec$ order and each edge once as $a \prec b$, sorted, only for pairs that share an attribute; a Bev that shares nothing stays as an isolated node (a singleton family). Duplicate or empty ids, an empty attribute, an unknown Bev and a repeated (Bev, attribute) row are rejected. The result is a valid graph for every function above. There is no constant: which attributes to project on is the caller's choice (`attributeProjection`, the header rule `attributeProjectionRule`, with cases in `network.vectors.json`).

The sync projects on `flavor:`*base* and `note:`*token* (the bases and notes of the flavour profiles), leaving out the catch-all base `other`, which is not a flavour, and stores as $\phi(b)$ the Louvain community of b in G_F, identified by its hub in G_F (Definition 10.28). The cost is $O(\sum_x \deg(x)^2)$, small at the catalogue's size. In the example of two flavour groups of three Bevs, each pair sharing three attributes of degree 3 ($w^F = 3/\ln 3 \approx 2.7307$) and one cross pair sharing a rare note of degree 2 ($1/\ln 2 \approx 1.4427$), Louvain over G_F returns the two groups.

Why families and not similarity communities (measured 2026-11-10). On the live graph (385 Bevs, 4,653 stored pairs), Louvain over G itself mostly rediscovers the root categories: the share of Bevs in their community's most common category (the category purity) is 0.84 at $\gamma = 1$ and 0.97 at $\gamma = 3$, because (25) is dominated by its taxonomy terms. Such a community would tell `recommend()` little that the category does not. Louvain over G_F gives ten cross-category families (citrus and lemon-lime 50, berry 49, tropical 47, tea 46, orange, grape and apple 41, cola 36, coffee 34, root beer, ginger and cream 32, mineral and herbal 29, hop and wine 20), with category purity 0.47; 61% of projected pairs cross root categories against 14% of the stored pairs. The partition hardly depends on the weighting: Rand index 0.97–0.99 between (35), the unweighted projection and a base-only one. So the stored community is the taste family, and the bridge stays a property of the similarity graph: a Bev whose similar Bevs lie in other families.

11 Implementation notes, divergences and versioning

Implementation note 11.1 (Where the maths runs). `@bevi/ranking` is pure TypeScript with no I/O or clock; Postgres holds *twins* of the formulas that touch stored data (re-spacing, statistics, Bayesian score, rank points, streaks) and the two are held equal by the golden vectors in `shared/fixtures` (one `*.vectors.json` file per formula), regenerated with `npm run vectors` on every `RANKING_VERSION` bump. Predictions and recommendations run in the client over the snapshot returned by `get_prediction_snapshot()` (decided 2026-10-12; revisit at about 200 users or a 1 MB snapshot).

Implementation note 11.2 (Prior mean; resolved 2026-11-01). `bayes_prior(p_category_id)` (20261019000200, #112) is the SQL twin of `globalPrior` and `categoryPrior`: a category's own mean at 30 rankings, else the global mean, and 5.0 on an empty system; `get_bev_leaderboard` has used it since then, so a Water on the water board shrinks toward Water's mean. Three gaps stayed open until 20261101000300 (A-14 follow-ups, #150). `get_feed`, `get_user_feed`, `get_explore` and `bev_of_period_at` read `coalesce((select mean_score from global_stats), 0)`, a prior of 0 on an empty system; they now call `bayes_prior(null)` once per call through a materialised CTE. The category branch was live and exact while the global branch read `global_stats.mean_score`, a `numeric(5,3)` refreshed every 15 minutes; the category branch now reads `category_stats`, a per-category mean and count that `refresh_global_stats` writes in the same transaction as `global_stats` at the same precision, so the two branches share freshness and precision (and the per-call `rankings` ⋈ `bevs` aggregate is gone). And `refresh_global_stats` wrote `bev_stats.bayes_score` only for rows with `rating_count > 0`, so a Bev whose last ranking was removed kept its old figure until it was ranked again; `refresh_bev_stats` now writes the column itself on every change, null at `rating_count = 0`, and the refresh job rewrites every row.

Implementation note 11.3 (Name order; resolved in 1.3.0). `get_bev_leaderboard` has ordered by `bayes desc, rating_count desc, name collate "C", id` since 20261019000200, the order of Section 5; the earlier "Bayesian score only" divergence is closed, and `user_category_stats.affinity_z` has had a writer since 20261019000100 (A-11). The last difference, on the TypeScript side, is closed in `RANKING_VERSION` 1.3.0 (M-28). `leaderboardOrder` compared names with JavaScript `<`, which orders UTF-16 code units, whereas `collate "C"` orders code points, so the two disagreed when one name had a character outside the Basic Multilingual Plane and the other a character in U+E000–U+FFFF at the same position (an emoji against a private-use or specials character). `compareCodePoints` in `bayesian.ts` now compares a code point at a time, `starter.ts` reuses it for its tie-breaks, and `bayesian.vectors.json` gained a case with such a pair. Every catalogue name is ASCII, so no existing order changed.

Implementation note 11.4 (Streak history in SQL; resolved 2026-11-01). Until 20261101000200, `touch_streak` (20261011000700) recomputed D_u from `rankings.ranked_at`. A re-rank deleted the row and re-inserted it with `ranked_at = now()`, and a removal deleted it, so the original day left D_u whenever it held no other ranking; `feed_events` cascaded on the same delete, and `comparisons` rows exist only when a question was asked. Section 9 counts ranking events, so after such a re-rank or removal the SQL current and longest streaks could be shorter than the specification's. Since 20261101000200 (A-14 follow-ups, #149) `submit_ranking` and `rerank_bev` append to `ranking_events` (user, Bev, kind, timestamp), nothing deletes from it, and `touch_streak` reads D_u from it through the pure `streak_from_days(date[], date)`, which pgTAP `18_ranking_events` pins against every day-period case of `streak.vectors.json`, including the two event cases written in 1.4.0 (a re-ranked day and a removed day that must still count). The migration backfilled the log from `rankings`, the `comparisons` submissions, `feed_events` and the run each `streaks` row recorded; a past longest run that included a question-less ranking later moved or removed is the one history no source can recover.

Implementation note 11.5 (Comparison log; resolved 2026-11-01). Until 20261101000200, `submit_ranking` and `rerank_bev` checked insertIndex $\in [0, \texttt{expectedBucketSize}]$ and the list length, and stored every comparison with its tie flag without re-checking Proposition 2.6: the log was a record, not a constraint. Since then `assert_comparison_log` checks every triple against the list as it stands before the insert (after the old row's removal on a re-rank), before anything is written: exactly one side of a triple is the new Bev and the other is in the list; a tie requires index $=$ pos $+ 1$, the new Bev winning requires index $\leq$ pos, the incumbent winning requires index $\geq$ pos $+ 1$, with pos the compared Bev's 0-based position. A compared Bev outside the list means the client searched a stale list and raises `BEVI_STALE_BUCKET` (a `conflict`, so the app restarts placement on a fresh list, as it does for a length mismatch); a malformed triple or an index that disagrees with an answer is a client bug and raises `BEVI_VALIDATION`. The rule was checked against every `placement` and `anchoredPlacement` golden case before it was written in SQL (48 cases, 153 comparisons, 0 failures).

Implementation note 11.6 (Stored precision). Stored scores are exact (`numeric(4,2)` is cents/100). Aggregates are rounded on storage: `bev_stats.avg_score`, `score_stddev` and `bayes_score` to two decimals; `user_stats`, `user_category_stats` and `global_stats.mean_score` to three. `bayes_score` is computed from the exact `sum_score`, not from the rounded mean. Since 20261101000300 every live Bayesian figure (the leaderboard, the feed, Explore, the Bev of the period) is rounded to two decimals at the RPC boundary, half away from zero like the `numeric(4,2)` store, so the live and stored paths agree; the leaderboard still orders by the unrounded value. The prediction snapshot computes user statistics live, so the client's z-scores are exact. The golden vectors are compared unrounded at 10^{-6} in TypeScript and at column scale in pgTAP. Since 1.4.0 the `bayesScoreStored` cases of `bayesian.vectors.json` pin the stored figure itself: inputs whose exact result lies on a half cent (5.925, 5.945, 5.895), with the value `numeric(4,2)` must hold computed by integer arithmetic, because rounding the double (e.g. 5.92499…) would give the other cent; pgTAP `20_vectors` runs them against the store's own expression, which is exact because the arithmetic is `numeric` throughout.

Implementation note 11.7 (What pgTAP checks against the vectors). `08_taste_profile` pins one case of `user-stats.vectors.json`; `09_leaderboard_parity` pins `bayes_prior` and the full leaderboard order on its own fixture, including the `collate "C"` tail; `07_bev_of_period` pins Section 10.3 on a fixed clock; `10_explore` pins one value of (28). Since 2026-11-01 (A-14 follow-ups) `20_vectors.test.sql`, generated from the fixtures by `scripts/vectors/build-pgtap.ts` and kept current by a CI check, runs every `storedScore` case against `ranking_score`, the `populationStats` cases against `stddev_pop`, the `bayesianAverage` and `bayesScoreStored` cases against `bayes_score`

and its stored cast, and the `rankPoints` cases against `rank_points`; `18_ranking_events` runs every day-period case of `streak.vectors.json` against `streak_from_days`. Still untested in SQL: `respaceBucket`, `rankUsers`, `categoryPrior` and `leaderboardOrder` (the last two have fixture-based coverage in 09), and the two weekly streak cases.

Implementation note 11.8 (Exploration slots). The development document lists the three slots for a 20-item list as 4, 9, 15; the implemented even-spacing rule (20) gives 4, 9, 14. The code is the specification here.

Version	Date	Change
1.0.0	2026-10-09	First vectors: bands 7/4, placement, exact cents, statistics, Bayesian average, rank points, streaks
1.1.0	2026-10-11	Prediction snapshot, Layers 1–2, blend, exploration, starter set
1.2.0	2026-10-14	Bev-graph edge weights: tree (23), profile (24) and embedding similarity, w (25), top-K_E selection, as `edges.vectors.json` (M-24); earlier vectors unchanged apart from the header
1.3.0	2026-10-24	Taste formulas of Section 6.4 (M-28): Pearson over the common subset (12), taste vectors (13), attribute similarity, the blend (14), neighbour selection and reliability, as `graph.vectors.json`. Layer 2 gains `userSimilarity: 'pearson'`, default `'cosine'`, so live predictions are unchanged; w is unchanged. Fix: leaderboard and starter-set names now compare by Unicode code point as Section 5 specifies, which resolves Implementation note 11.3 (they compared UTF-16 code units, which put astral characters before U+E000–U+FFFF), with one new `leaderboardOrder` vector; ASCII names, hence every catalogue name, order as before. Earlier vectors unchanged apart from the header
1.4.0	2026-10-31	Layer 1 edges mode (M-27 (a)): `bevSimilarity: 'edges'` reads the stored weights of `bev_edges` from the snapshot's optional `edges` field as (9), 0 for a pair without an edge; a snapshot without the field falls back to (5). The default `'content'` is byte-identical to 1.3.0. New golden cases, no other formula change: `layer1` in `edges.vectors.json` (an edge, no edge, either orientation, the missing field, the default ignoring the field); `bayesScoreStored` and the Example 5.3 case in `bayesian.vectors.json`; two `streakFromTimestamps` cases written as ranking events. w is unchanged. Earlier vectors unchanged apart from the header
1.5.0	2026-10-31	Anchored placement (M-21, Section 10.1) in the package: `placementTiers` (26) (same leaf 1, same root 2, stored edge across roots 3, else none; the taxonomy first, a weight only ever adds tier 3) and `startPlacement` with an optional `tiers` array, running Algorithm 1 inside the existing state machine (`anchored` on the state, both ranges on every history step, undo across the phase boundary, the same `PlacementResult`). Without `tiers`, and with tiers that select no anchor, the machine is byte-identical to 1.4.0: 252,144 states, prompts, flags and results compared in lockstep against the 1.4.0 source, 0 differences. New golden cases in `placement.vectors.json`, with two header rules (`placementTiersRule`, `anchoredPlacementRule`): `placementTiers` over D-04 §3's pairs, and `anchoredPlacement` scripts (the figure above, ties in either phase, undo across the boundary, re-rank, each tier alone, the two fallbacks, a 20-Bev oracle) recording the questions asked with their phase, the anchors, `maxSteps` and the result. No SQL

A Constants

All values from `packages/ranking/src/constants.ts`, `RANKING_VERSION` 1.8.0.

Constant	Value	Role
`BANDS`	$[0, 4]$, $[4, 7]$, $[7, 10]$	Score bands for DISLIKED, FINE, LOVED; the first two exclude their top edge
`SIGMA_FLOOR`	1.0	Floor on any standard deviation used for z-scores
`MIN_N_USER`	3	Below this a user's z-scores use global statistics; predictions withheld
`MIN_N_CAT`	3	Below this `effectiveCategoryStats` falls back to the user's overall statistics; exported and in the vectors; the product shows raw category means (Section 10.2), and it is kept as a display helper and for the evaluation's per-category baseline (M-22)
`GLOBAL_FALLBACK`	$(5.0, 2.5)$	Only when the system has no rankings
`K_AFF`	2	Shrinkage of category affinity
`K_G`	3	Shrinkage of a Bev's normalised global score
`BAYES_C`	3	Prior weight of the Bayesian average (raise to 5 when median $n_b > 10$)
`BAYES_CATEGORY_PRIOR_MIN`	30	Rankings a category needs before its own mean is the prior
`MIN_RATERS`	2	Leaderboard eligibility
`SIM_WEIGHTS`	0.40 / 0.30 / 0.20 / 0.10	Category, brand, flavour tokens, price tier
`SIM_MIN`	0.1	Pairs below this are dropped; applies to sim_E (9) too
`PRICE_TIER_SPAN`	3	Largest $\lvert\text{tier} - \text{tier}'\rvert$ (tiers 1–4)
`LAYER1_WEIGHTS`	0.40 / 0.40 / 0.20	Affinity, content, global
`CONF1_USER_N`, `CONF1_MASS`	10, 4	Layer 1 confidence saturation
`CF_K`	10	Neighbours per prediction
`CF_MIN_COMMON`	2	Minimum common items for a non-zero user similarity
`CF_SIGNIFICANCE`	5	Significance weighting of user similarity
`LAMBDA_MAX`, `LAMBDA_USER_N`	0.7, 15	Blend weight ceiling and user-history saturation
`CONFIDENCE_LOCK`	0.25	Below this the UI shows a lock
`EPSILON`	0.15	Share of the list given to exploration
`JITTER_SD`	0.15	Ordering-only jitter (z-units)
`RECOMMEND_N`	20	Default list length (API caps at 50)
`STARTER_K`, `STARTER_CORE`	10, 6	Onboarding set size and shared core
`STARTER_MAX_PER_CATEGORY`, `STARTER_MIN_CATEGORIES`	2, 5	Spread of the onboarding set
`RANK_POINTS_WEIGHTS`	1 / 3 / 2	Bevs ranked, current streak, adopted Bevs created
`STREAK_PERIOD`	day	Daily streak (weekly is a constant change)

Constant	Value	Role
EDGE_WEIGHTS	0.25 / 0.15 / 0.30 / 0.30 / 0	Tree, content, embedding, profile, co-preference (25)
EDGE_TOP_K, EDGE_MIN_WEIGHT	20, 0.3	Edges kept per Bev (K_E) and their minimum weight
PROFILE_WEIGHTS	0.40 / 0.15 / 0.10 / 0.05 / 0.15 / 0.15	Base, carbonation, dairy, alcohol, caffeine, sweetness (24)
PROFILE_LEVEL_SPAN	3	Largest caffeine or sweetness difference
TASTE_K	2	Shrinkage K_T of a taste-vector entry (13)
TASTE_MIN_ATTR_BEVS	2	Catalogue Bevs an attribute needs to be kept
TASTE_ALPHA	0.5	Pearson weight α of the blend (14) when $m \geq 2$
TASTE_SHARED_MIN, TASTE_MIN_SHARED_ATTRS	0.2, 2	$\lvert t \rvert$ for a shared attribute; bridging attributes an attribute-only pair needs
TASTE_BRIDGE_KINDS	brand, flavour	Attribute kinds that bridge (categories do not)
TASTE_SIM_FLOOR, TASTE_NEIGHBOUR_K	0.1, 25	Least similarity kept and neighbours kept per user in `user_neighbors`
NEIGHBOUR_RELIABILITY_N	15	N_0 of the reliability weight $\min(1, n_v/N_0)$ (chosen; the doc leaves it open)
PLACEMENT_TIER	1 / 2 / 3	Anchored placement tiers (26): same leaf, same root, stored edge across roots (the floor is EDGE_MIN_WEIGHT)
PAGERANK_DAMPING	0.85	PageRank damping d (31)
PAGERANK_TOLERANCE, PAGERANK_MAX_ITERATIONS	10^{-10}, 200	PageRank stops when the L_1 step is below τ; T is a safety rail, never reached at the defaults ($2d^t < \tau$ for $t > 146$)
LOUVAIN_RESOLUTION	1	Modularity resolution γ (32)
LOUVAIN_TIE_EPSILON	10^{-12}	Margin a Louvain candidate must beat the best gain by (Definition 10.25)
DIVERSITY_PENALTY	0.2	z-units per earlier pick from the same taste family (21); option off, unvalidated
EXPLORE_BRIDGE_WEIGHT	1	Bridge multiplier β of the community exploration weight (22); option off, unvalidated

B Symbols

Symbol	Meaning
U, B, C	Users, Bevs, categories; $\mathrm{cat}(b)$ the category of b
$R(u), R^{-1}(b)$	Bevs ranked by u; users who ranked b
$n_u, n_b, n_{u,c}$	Counts of rankings by user, of a Bev, by user within a category
$s_{u,b}$	Stored score of u for b (two decimals)
$\sigma(i, n)$	Score of position i in a bucket of n (eq. 1)
μ_u^*, σ_u^*	Effective mean and (floored) standard deviation of u
$z_{u,b}$	Normalised score of u for b
$\mathrm{aff}_{u,c}$	Category affinity (eq. 2); g_b normalised global score (eq. 3)
bayes_b	Bayesian leaderboard score (eq. 4); m its prior mean, C its weight
$\mathrm{sim}(b, b')$	Content similarity; $\mathrm{sim}'(u, v)$ significance-weighted user similarity
$\rho, \rho', t_u, \mathrm{sim}_A, \mathrm{sim}_B$	Pearson and shrunk Pearson user similarity, taste vector, attribute and blended user similarity (Section 6.4)
$\hat{z}^{(1)}, \hat{z}^{(2)}, \hat{z}$	Layer 1, Layer 2 and blended predictions in z-units; $\hat{s}$ in score units
$\lambda, \mathrm{supp}, \mathrm{conf}$	Blend weight, neighbour support, confidence
$\mathrm{leaf}(b)$	The Bev's leaf category, or its root when it has none (Section 10.1)
$\mathrm{sim}_T, \mathrm{sim}_{\mathrm{cop}}, w$	Tree similarity, co-preference and Bev-graph edge weight (Section 10.1)
sim_E	Stored edge weight as Layer 1's similarity in edges mode (9)
$\mathrm{tier}(b, \ell)$	Tier of a bucket Bev in anchored placement (26): 1 same leaf, 2 same root, 3 stored edge, or none
A, G, k, g	Anchors, gap, and their sizes in anchored placement; p_j the position of a_j in L, j^* the anchor slot Phase 1 returns
$\mathrm{hot}(p, t)$	Time-decayed popularity of a post (eq. 28)
$s_i, m, \prec$	Strength (weighted degree) of a Bev in the Bev graph, its total edge weight, code-point order (Section 10.9)
$\pi, Q_\gamma, P_i, h(C)$	PageRank (31), modularity (32), participation coefficient (34), community hub
$G_F, w^F, \phi(b)$	Flavour projection and its Adamic–Adar weight (35); the taste family of b (Section 7)
δ, β, Φ_u	Diversity penalty (21), bridge weight (22), the families charted by u